

Identity API v7

Installation

Docker

1. Follow the steps [here](#) ↗ to install Docker Engine on your host machine.
2. Verify your installation by running `docker run hello-world`

Identity API container

Ensure you have Docker Engine installed and that you're logged in with the Docker user we've given access to in our repo. Then, run the following to download the image.

```
docker pull containers.paravision.ai/identity/identity:<tag>
```

Ports

Expose port 5656 on your machine to communicate with the API container externally (or another port if you choose to map differently).

Database

If you wish to use Milvus (for large scale data), follow [Milvus/MySQL setup guide](#).

For the regular (PostgreSQL) variant, follow [Postgres setup guide](#).

Run Container

With Milvus database, you can run the following to start the API with appropriate port mapping:

```
docker run -dt --rm --name identity-api \
  --link <MYSQL_CONTAINER_NAME> --link <MILVUS_CONTAINER_NAME> -p
5656:5656 \
  -e PV_SCORING_METRIC=match_score \
  -e PV_MYSQL_URI="<DB_USER>:
<DB_PASSWORD>@tcp(<MYSQL_CONTAINER_NAME>:3306)/<IDENTITY_DB_NAME>" \
  -e
PV_MILVUS_URI="tcp://<MILVUS_CONTAINER_NAME>:19530/<MILVUS_COLLECTION_P
REFIX>?<MILVUS_PARAMETERS>" \
  containers.paravision.ai/identity/identity:<tag>
```

For the PostgreSQL database:

```
docker run -dt --rm --name identity-api \
  --link <DB_CONTAINER_NAME> -p 5656:5656 -e
PV_SCORING_METRIC=match_score \
  -e PV_POSTGRES_URI=postgres://<DB_USER>:
<DB_PASSWORD>@<DB_CONTAINER_NAME>:<DB_PORT>/<DB_NAME>?sslmode=disable \
  containers.paravision.ai/identity/identity:<tag>
```

If you wish to enable the optional HTTP REST interface, see instructions for [HTTP usage](#).

Options

The following options can be configured via environment variables when running the container. For example, `-e PV_SSL=on`.

Consult the relevant database setup instructions page for an explanation of `OPTIONS` in `*_URI` variables.

PV_POSTGRES_URI

URI for the PostgreSQL connection. **Required for the PostgreSQL variant**, default: *empty*.

Syntax: `postgres://USER:PASSWORD@HOST:PORT/DATABASE?OPTIONS`

PV_MYSQL_URI

URI for the MySQL connection. **Required for the Milvus variant**, default: *empty*.

Syntax: `USER:PASSWORD@tcp(HOST:PORT)/DATABASE?OPTIONS`

PV_MILVUS_URI

URI for the Milvus connection. **Required for the Milvus variant**, default: *empty*.

Syntax: `tcp://HOST:PORT/COLLECTION_PREFIX?OPTIONS`

PV_HTTP_INTERFACE

When set to `on`, enables [HTTP Interface](#). Default: `off`.

PV_USER

Username for basic authentication. Non-empty value enables HTTP basic authentication in HTTP interface and in gRPC. Cannot contain any colons or control characters (see [RFC 7617 ↗](#)). Default: *empty*.

 Providing `PV_USER` enables basic authentication in HTTP *and* in gRPC. For more details on authenticating in HTTP refer to [HTTP Usage](#) and for more details on authenticating in gRPC refer to [Usage](#).

PV_PASSWORD

Password for basic authentication. Cannot contain any control characters (see [RFC 7617 ↗](#)). Default: *empty*.

PV_PASSWORD_FILE

Path inside the container to a file with password for basic authentication, to be used instead of `PV_PASSWORD`. Default: *empty*.

PV_SSL

When set to `on`, enables SSL/TLS encryption for gRPC (and also HTTP if interface is enabled). Requires `PV_SSL_CERT` and `PV_SSL_KEY` files to exist within the container. You can bind mount them from the host using `-v` / `--volume` option in `docker` command line, e.g. `-v HOST_PATH:/server.crt -v HOST_PATH:/server.key`. Only connections using TLS 1.2 or higher are supported. Default: `off`.

PV_SSL_CERT

Path inside the container to the SSL Certificate. Default: `/server.crt`.

PV_SSL_KEY

Path inside the container to the SSL Private Key. Default: `/server.key`.

 Refer to the [tutorial](#) for more details on SSL set-up.

PV_SCORING_METRIC

Metric to be returned when looking for matching faces and for thresholds when checking for similar already-existing embeddings. Please read about [choosing the score](#) for more details. Can be set to either `match_score` or `confidence`, default: `confidence`.

PV_SCORING_WEIGHT, PV_SCORING_BIAS

Weight and Bias parameters are floating point numbers used for calculating matching functions. Can only be set together. When not set, the default values for the selected matching function will be used. Default: *empty*.

PV_CONNECTION_AGE_MS

An integer denoting maximum lifespan of one connection to the server, in milliseconds. Accepted values range from `1` to `2^31 - 1`, where the latter represents infinity. There is a 10 seconds grace period that allows already started requests to complete. Our clients will automatically re-connect to the server when the current connection gets closed, so using this setting does not require any changes client-side. This setting can be useful when using client-side load-balancing. Default: `2^31 - 1`.

PV_PROMETHEUS_METRICS

When set to `on`, enables [Prometheus](#) metrics in the Processor container. Default: `off`.

Metrics will be exposed on the port `8090`, under the default `/metrics` path. Currently only one metric, `pv_identity_request_duration_seconds` is exposed, which is a [Histogram](#) of request durations to Identity API. It has two labels: `method` refers to the appropriate API method, while `status_code` is a human-readable name of one of the [GRPC status codes](#).

As an example, to calculate the number of successful lookups per second over the last 5 minutes, you can then use this query in PromQL (but remember to poll metrics often enough so that the result is meaningful):

```
rate(pv_identity_request_duration_seconds_count{method="Lookup",  
status_code="OK"}[5m])
```

 Refer to the [tutorial](#) for more details on Prometheus metrics.

PV_PROMETHEUS_METRICS_MAX_BUCKET

A positive integer denoting the upper bound of the largest bucket of the [histogram](#) of request durations. Other buckets will be calculated automatically, with their upper bounds spread out exponentially. You may want to adjust this value if you

expect that request times will often exceed the default value in normal operation (otherwise calculation of quantiles in Prometheus may be inaccurate), see the [tutorial](#) ↗ for details. Default: `10`.

PV_LOG_THROUGHPUT_PERIOD

A positive integer - when set, every `PV_LOG_THROUGHPUT_PERIOD` seconds Identity server will log the number of succesful lookup requests per second in the that time period. Requires enabling `PV_PROMETHEUS_METRICS` as well. Default: *empty*.

Choosing the scoring metric

Starting with [6.0.0](#) match score is supported - a new metric for scoring embeddings' similarity. However, by default, confidence (now obsolete) will be used. The metric to be used by Identity API can be set using `PV_SCORING_METRIC` environment variable.

For both scoring metrics, the greater the value, the more likely it is that the two embeddings represent the same person. Moreover:

- confidence is a real number between 0 and 1,
- match score is an integer value between 0 and 1000.

In both cases they're represented with a floating-point type.

If your use case requires custom weight and bias values, they also can now be configured with an environment variables `PV_SCORING_(WEIGHT|BIAS)`.

Client

C#

1. Add IdentityCSharpClient nuget source

```
dotnet nuget add source https://nuget.fury.io/<token>/<username>/
```

where `<token>` and `<username>` are your credentials to the Gemfury repository we provide you.

2. Add IdentityCSharpClient nuget package to your project. Make sure you are in the directory that has your csproj file

```
dotnet add Application.csproj package IdentityCSharpClient -v 6.0.0
```

where `Application.csproj` is the csproj file for your application.

C++

1. Download pre-built libraries for your system

- Repository: `https://<user>:<password>@paravision.mycloudrepo.io/repositories/identity-cpp-client/`
- Linux 64bit build: `paravision-identity-client-v6.0.0-linux-x64.tar.gz`
- Windows 64bit, MSVC v140+ (Visual Studio 2015+): `paravision-identity-client-v6.0.0-windows-x64-msvc-v14x.zip`

Where `<user>` and `<password>` are your credentials to the repository we provide you.

2. Unpack library files (`lib/`, and `bin/` on Windows) and headers(`include/`) to desired directory and setup linking in your project

Go

1. Configure Go to use Gemfury proxy.

```
export GOPROXY=https://<token>@go-proxy.fury.io/<username>/  
export GOPRIVATE="golang.paravision.ai/*"  
export GONOPROXY=none
```

where `<token>` and `<username>` are your credentials to the Gemfury repository we provide you.

2. Add our client package to the list of dependencies in the `go.mod` of your project:

```
require golang.paravision.ai/identity-go-client/v6 v6.0.0
```

3. Run `go mod download` in your Go project.

Java

1. Configure your dependency manager to use the Maven repository at `https://maven.fury.io/paravision`, with HTTP basic auth, using the repository token we give you as username, and `NOPASS` as the password.

For example, with Gradle you can use:

```
repositories {
    jcenter() // Not required, but our repository does not mirror
              // any packages, so you'll likely need some other
              // base repository.

    maven {
        name = 'paravision'
        url = 'https://maven.fury.io/paravision'

        authentication {
            basic(BasicAuthentication)
        }

        credentials {
            username = "<paravision_token_here>"
            password = "NOPASS"
        }
    }
}
```

2. Add the `identityclient:6.0.0` dependency from `ai.paravision` group to your project.

A fat JAR is also available. It bundles all transitive dependencies together with Paravision library. To use it, append `.fat` to version number, for instance:

```
identityclient:6.0.0.fat
```

Continuing the Gradle example:

```
dependencies {  
    implementation 'ai.paravision:identityclient:6.0.0'  
}
```

3. The resources provided by Identity Java client are available in the `ai.paravision.identity` package. Thus your Java source files might use:

```
import ai.paravision.identity.*;
```

Node

1. Add our client package to the list of dependencies in the `package.json` of your project:

```
"@paravision/identity-js-client": "6.0.0"
```

2. Add the following to your `.npmrc` file:

```
@paravision:registry=https://npm.fury.io/<username>/  
registry=https://registry.npmjs.org/  
  
//npm.fury.io/<username>/:_authToken=<token>
```

where `<token>` and `<username>` are your credentials to the Gemfury repository we provide you.

3. Run `npm install` in your node project.

Python

1. Install identity client package

```
python3 -m pip install \
  --no-cache-dir \
  --extra-index-url https://<user>:
<pass>@paravision.mycloudrepo.io/repositories/identity-python-client
\
  paravision-identity-client==6.0.0
```

where `<user>` and `<password>` are your credentials to the repository we provide you.

Ruby

Install Identity client package `paravision-identity-client` from the `https://gem.fury.io/paravision/` source, authorizing with the credentials to the Gemfury repository we provide you.

RubyGems

Simply install the `paravision-identity-client` gem

```
gem install paravision-identity-client -v 6.0.0 --source
https://<paravision_token_here>@gem.fury.io/paravision/
```

Bundler 1.8+

Add `paravision-identity-client` gem to your `Gemfile`

```
gem 'paravision-identity-client', '6.0.0', source:
'https://gem.fury.io/paravision/'
```

and configure Bundler to use your repository token

```
bundle config https://gem.fury.io/paravision/
<paravision_token_here>
```

Finally, install the client package

```
bundle install
```

Bundler pre-1.8

Add `paravision-identity-client` gem to your `Gemfile`

```
gem 'paravision-identity-client', '6.0.0', source: 'https://#{ENV['FURY_AUTH']}@gem.fury.io/paravision/'
```

and define `FURY_AUTH` environment variable before using Bundler

```
export FURY_AUTH=<paravision_token_here>
```

or using `bundler config` to globally specify the auth info

```
bundle config https://gem.fury.io/paravision/  
<paravision_token_here>
```

However it is possible for Bundler to occasionally put your token into `Gemfile.lock`. It is safe to remove the token and commit just the URL without credentials into your SCM

```
GEM  
  remote: https://gem.fury.io/paravision/  
  ...
```

Finally, install the client package

```
bundle install
```

Database (Postgres)

Paravision distributes four flavors of Postgres DBMS, used to store identity information for Identity API.

All flavors include modification to allow connections from Docker's inter-container ip ranges and `cube` extension with raised `CUBE_MAX_DIM` value, but not all support embeddings from precision models.

Flavor name	Postgres version	Supported embedding size	Note
(normal)	12.12	up to 512	
<code>expanded</code>	12.12	up to 1024	increased block size to 16kB
<code>ha</code>	11.17	up to 512	based on Bitnami's postgresql-repmgr image, with support for high-availability through postgresql-ha Helm chart (see tutorial)
<code>ha-expanded</code>	11.17	up to 1024	like <code>ha</code> , with increased block size to 16kB

Set up Postgres database

 **Warning:** Launching the containers like in this guide will result in Postgres data being stored inside the container - which means that **all data will be lost** once the container is deleted. Consider using Docker volumes or other such mechanism to persist your data - read [Docker documentation about storage](#) to learn more.

1. Run and setup our `postgres` Docker image

Ensure that you're logged in with the Docker user we've given access to in our repo. Then, execute the following to download our Postgres image from DockerHub and run.

```
# Run the postgres container
docker run -d --name <DB_CONTAINER_NAME> -e POSTGRES_USER=<DB_USER> -e
POSTGRES_PASSWORD=<DB_PASSWORD> -e POSTGRES_DB=<DB_NAME>
containers.paravision.ai/postgres/postgres:12.12
```

2. Generate database tables

Create the necessary tables with the following command.

```
# Set env var with full database URI for Identity API (ssl disabled for
demo purposes)
export PV_POSTGRES_URI=postgresql://<DB_USER>:
<DB_PASSWORD>@<DB_CONTAINER_NAME>:<DB_PORT>/<DB_NAME>?sslmode=disable

# Run schema migrations
docker run --rm --link <DB_CONTAINER_NAME> --entrypoint /migrate
containers.paravision.ai/identity/identity:<tag> -source
file://migrations/postgres --database $PV_POSTGRES_URI up
```

 **Note:** The `PV_POSTGRES_URI` environment variable used here should also be set as an environment variable when running the Identity API.

Release notes

 To migrate from the previous version, [create a backup of an old database and restore the data from there ↗](#). Re-using database storage backend might cause data errors.

Standard

12.12

Aug 15, 2022

- Upstream upgrades.

12.11

May 26, 2022

- Upstream upgrades.
- Both standard and `-expanded` flavors are now a custom compilation of PostgreSQL based on [Alpine ↗](#) image.
- Removed some unused packages from the container.

High Availability with repmgr

11.17

Aug 15, 2022

- Upstream upgrades.

11.16

May 26, 2022

- Upstream upgrades.
- Both `-ha` and `-ha-expanded` flavors are now a custom compilation of PostgreSQL based on Bitnami's [postgresql-repmgr ↗](#) image.
- Removed some unused packages from the container.

Database (Milvus/MySQL)

Databases setup

You can setup Identity API to use [Milvus](#) for storing and searching the embeddings, in conjunction with a MySQL database for storing data about identities and Milvus metadata. This vastly improves search performance and is suitable for bigger datasets, but the setup is a bit more complex.

Below you can find a basic guide on how to set up MySQL and Milvus running in Docker containers for use with Identity. Make sure to also visit [Milvus configuration page](#) for explanations with regards to configuration options, performance tuning and more. When in doubt, consult official installation instructions.

 **Warning:** Launching the containers like in this guide will result in MySQL and Milvus data being stored inside the container - which means that **all data will be lost** once containers are deleted. Consider using Docker volumes or other such mechanism to persist your data - read [Docker documentation about storage](#) to learn more.

Set Up MySQL DB

 **Note:** For MySQL, version 5.7 is required.

You will need to set up **two** MySQL databases (one for Identity API itself, another for Milvus internal metadata storage).

 This guide hosts the two databases on the same installation and uses the same user to access both, but you can host them on separate MySQL hosts and/or use separate users if you wish.

1. Run and setup the MySQL Docker image

Execute the following to run the official MySQL docker image from Docker Hub.

```
# Run the MySQL container
docker run -d --name <MYSQL_CONTAINER_NAME> -e MYSQL_ROOT_PASSWORD=
<ROOT_PASSWORD> mysql:5.7
```

2. Create required databases

Execute the following to run a command line client to connect to your MySQL installation:

```
# Run the MySQL command line client
docker run -it --rm --link <MYSQL_CONTAINER_NAME> mysql:5.7 mysql -h
<MYSQL_CONTAINER_NAME> -u root -p
# Enter password: <ROOT_PASSWORD>
```

In the client, run the following to create the two required databases and an user, and grant relevant access.

```
CREATE DATABASE <IDENTITY_DB_NAME>;
CREATE DATABASE <MILVUS_DB_NAME>;
CREATE USER <DB_USER>@'%' IDENTIFIED BY '<DB_PASSWORD>';
GRANT ALL PRIVILEGES ON <IDENTITY_DB_NAME>.* TO <DB_USER>@'%' ;
GRANT ALL PRIVILEGES ON <MILVUS_DB_NAME>.* TO <DB_USER>@'%' ;
```

Then, exit the client by typing `\q`.

3. Generate database tables

Create the necessary tables for the Identity API DB with the following command.

```
# Set env var with full database URI for Identity API (ssl disabled for
demo purposes)
export PV_MYSQL_URI="<DB_USER>:
<DB_PASSWORD>@tcp(<MYSQL_CONTAINER_NAME>:3306)/<IDENTITY_DB_NAME>"

# Run schema migrations
docker run --rm --link <MYSQL_CONTAINER_NAME> --entrypoint /migrate
containers.paravision.ai/identity/identity:<tag> -source
file://migrations/mysql --database mysql://$PV_MYSQL_URI up
```

 **Note:** The `PV_MYSQL_URI` environment variable used here should also be set as an environment variable when running the Identity API.

Set Up Milvus

 **Note:** For Milvus, version from the 1.x branch is required. This part of the guide uses Milvus 1.1.1, which is the latest 1.x version at the time of this writing. You can also check [official Milvus installation instructions](#).

1. Configure Milvus

Execute the following to download the default configuration file for Milvus:

```
wget https://raw.githubusercontent.com/milvus-
io/milvus/v1.1.1/core/conf/demo/server_config.yaml
```

Open `server_config.yaml` in a text editor and set `meta_uri` to point to the Milvus MySQL database to use for metadata:

```
general:
  timezone: UTC+8
  meta_uri: mysql://<DB_USER>:
<DB_PASSWORD>@<MYSQL_CONTAINER_NAME>:3306/<MILVUS_DB_NAME>
```

This is all that's required to start the container, but you may need to adjust other parameters in the config file for the purpose of performance tuning (see [Milvus configuration page](#)).

2. Run Milvus Docker image

Execute the following to run the Milvus Docker container:

```
docker run -d --name <MILVUS_CONTAINER_NAME> \  
  --link <MYSQL_CONTAINER_NAME> \  
  -v \  
  ${PWD}/server_config.yaml:/var/lib/milvus/conf/server_config.yaml \  
  containers.paravision.ai/milvus/milvus:1.1.1-20221021
```

Note: You can use the official image `milvusdb/milvus:1.1.1-cpu-d061621-330cc6` for Milvus setup but we provide slightly customized Milvus image under `containers.paravision.ai/milvus/milvus` for your convenience (see more on [Milvus configuration page](#)).

Connection string options

This part describes options that can be used inside `PV_MYSQL_URI` and `PV_MILVUS_URI` environment variables used in Identity API. You can use them to configure various aspects of the relevant database.

Syntax: `param1=value1¶m2=value2...`.

PV_MYSQL_URI

These parameters allow to customize Identity's connection with MySQL.

parameter	description
	Configures TLS / SSL encrypted connection to the server. You can use <code>true</code> to enforce

tls(bool, default: `false`)

encryption, `skip-verify` if your MySQL instance uses a self-signed or invalid certificate or use `preferred` to use TLS only when advertised by the server. Neither `skip-verify` nor `preferred` add any reliable security.

For more advanced use, should you need it, you can check official supported parameters in the SQL driver's [documentation](#).

PV_MILVUS_URI

These parameters describe the properties of the collection and index that are automatically created by Identity on first boot.

Warning: Choose these parameters wisely, as they cannot be changed without re-importing your Milvus collection from scratch (changing them and rebooting Identity will have no effect). To pick the right parameters, consult the performance tuning section in [Milvus configuration guide](#).

Note: Default parameters provide reasonable performance and accuracy for smaller datasets. If you expect to handle more than around 1GB of data, read the configuration guide carefully.

parameter	description
index_type (string, default: <code>IVF_FLAT</code>)	Milvus index type . Currently only <code>IVF_FLAT</code> , <code>IVF_SQ8</code> , <code>IVF_SQ8H</code> and <code>IVF_PQ</code> are supported.
index_file_size (int, default: <code>128</code>)	Size in MB of a single vector segment in Milvus . Corresponds to the same parameter of the underlying collection.
index_buckets (int, default: <code>1024</code>)	Number of clusters per one IVF index file, corresponds to <code>nlist</code> parameter from Milvus .

	When set to 0, no index will be used, and an exact search is performed.
max_vector_dimension (int, default: 128)	Maximum vector dimension used by Milvus. Smaller vectors passed to the API will be extended to this size.
pq_factors (int, default: 256)	For IVF_PQ index only: Number of factors of product quantization, corresponds to m parameter from Milvus
pq_nbits (int, default: 8)	For IVF_PQ index only: Number of bits in which each low-dimensional vector is stored, corresponds to nbits parameter from Milvus

Milvus configuration

You can configure Identity with [Milvus ↗](#) for storing embeddings, which enables faster lookups. This guide will help you the make the most out of it.

If you were using Postgres for vector storage before, and you wish to switch, keep in mind that you will have to **migrate all your data**.

Customized image

In `containers.paravision.ai/milvus/milvus:1.1.1-20221021` we provide slightly customized `milvusdb/milvus:1.1.1-cpu-d061621-330cc6` image. Main changes are:

- Installing latest available security updates.
- Adding support for `MILVUS_SERVER_CONF` environment variable to pass a full configuration file content instead of mounting the file from the host. This might be useful in some container orchestration solutions.

Milvus-specific function parameters

There are some Milvus-specific parameters in the API that you can use to influence performance (and accuracy) of the operations. They're simply ignored if Postgres database is used.

Flushing

Since Milvus uses an eventual consistency model, inserted vectors are generally not immediately available for search - they're only visible once data is flushed to disk (which by default happens every second in Milvus). Most commands in Identity are set to flush by default for compatibility with the regular variant, but if you want faster inserts and don't mind that new data isn't immediately searchable, you can set **flush** parameter to false.

Compact operation

When deleting vectors, Milvus doesn't actually free up the space immediately - entries are only internally marked as deleted but they still take up space. If you deleted lots of vectors recently and want to regain that space, you need to manually call a **compact** operation - we offer an API method to do this.

There is also a rare situation when a batch insert or delete operation fails - Identity will take care of the data consistency by keeping track of the vectors in the MySQL database (operation will be rolled back), but unwanted vectors may remain in Milvus. Compact also takes care of this situation, since it removes embeddings that aren't referenced by any identity - if you had a lots of failed operations recently, **compact** might be worth calling.

New search parameters

`buckets_limit` parameter was added to search-related functions, which corresponds to `nprobe` parameter in Milvus - more details below, but the bigger this parameter is, the slower but more accurate the searches are.

`scaling_factor` was added to control an implementation detail of Identity - most of the time you don't have to worry about it, especially if you don't have many faces per identity.

Since Milvus has no notion of identities and it can only keep embeddings, we store all vectors in multiple collections, one per group. This means that when searching for a specific number of closest identities, we don't know beforehand how many distinct identities will be returned (we can only ask for a specific number of faces, and some of them belong to the same identity). If the first search returns too few distinct identities, we repeat the search with `scaling_factor` times more embeddings than before, until the requested number of identities is found. If you have many faces per identity and find search too slow, you could try to improve your results by raising the `scaling_factor` (since it may reduce the amount of times Identity sends lookups to Milvus).

Accuracy and performance tuning

Note: These are just general guidelines, consult [Milvus documentation](#) for more detailed explanations.

cache_size

`cache_size` parameter from the `server_config.yaml` is crucial to get good query performance - it needs to be greater than total size of your data to avoid swapping data in and out of disk.

You can estimate the size of your data **per one group** by multiplying the number of vectors in a group by the number of dimensions (`max_vector_dimensions`), and then by 4 bytes (size of one floating-point number). You can also use the [Milvus sizing tool](#).

Note that each group is stored separately, so if an identity belongs to multiple groups, it will contribute to the data size multiple times.

`cache_size` directly corresponds to the memory usage of your Milvus installation, so you will need to have at least this amount of RAM on your machine. Additionally, building indexes in the background may use additional $2 * \text{index_file_size}$ MB of memory, which may be important if you set this value too high.

Approximate search

By default, an approximate nearest-neighbor search is used, using an [IVF_FLAT](#) index, but other quantization-based indexes mentioned in [Milvus documentation](#) are supported as well.

If you wish to use the brute-force search instead, set `index_buckets` to 0, which turns off the indexing completely. This will be slower for a large number of faces, but it will have 100% accuracy. (Milvus calls this a [FLAT index type](#)).

Index and search parameters

index_file_size

Milvus divides vectors into [segments](#) ↗, and each segment will have roughly `index_file_size` MB. Each segment will be indexed and searched separately, and Milvus will slowly fill out segments as you add vectors to it.

The bigger the `index_file_size`, the less segments in total - which improves search performance. On the other hand, as you add vectors, the segment that is not yet filled will not be indexed, so during search it will be brute-forced - which means `index_file_size` shouldn't be too large either, especially when you don't add vectors too often.

For smaller datasets, default value of `index_file_size` should be reasonable - for larger datasets, we recommend setting it so the number of vectors in one segment is no larger than 1 million vectors (so for example, if `max_vector_dimension` is 128, you could pick a value of `1 million * 4 bytes * 128`, which is not far from 512MB).

 **Note:** See also [Milvus FAQ](#) ↗ on this topic.

index_type

Identity uses `IVF_FLAT` ↗ index by default, but it's also possible to select other quantization-based indexes (`IVF_*`) mentioned in [Milvus documentation](#) ↗ by passing the appropriate name in the `index_type` parameter.

 **Note:** Only quantization-based indexes (`IVF_*`) are supported.

index_buckets, buckets_limit

Roughly speaking, when using IVF indexes, vectors in each Milvus segment are grouped into clusters, and only some of the neighboring clusters are searched instead of the whole dataset, which is what improves performance. (It can happen that the nearest vector is in a cluster you didn't check, which makes the search approximate).

`index_buckets` (`nlist` in Milvus) is a collection-specific parameter that determines the number of clusters. `buckets_limit` (`nprobe` in Milvus) is a search parameter, which determines the number of closest clusters to search. The bigger `buckets_limit` is, the slower the search, but the better accuracy.

⚠ As mentioned earlier, `index_buckets` is set only once on first Identity startup, and you can't change it afterwards unless you re-import vectors in Milvus from scratch manually. On the other hand, `buckets_limit` is a search parameter so you're free to experiment and change it per lookup.

The default values are reasonable for smaller datasets. Milvus recommends to set `index_buckets` to around $4 \times \sqrt{\text{number of vectors in a segment}}$, so e.g. when `index_file_size` is set to 512MB, and vector dimension is 128 you should set it to $4 \times \sqrt{512\text{MB} / (4 \times 128)}$, which is 4096.

ℹ **Note:** See also [Milvus FAQ](#) on this topic.

pq_factor, pq_nbits

ℹ **Note:** The following options affect `IVF_PQ` index only.

`IVF_PQ` index uniformly decomposes the original high-dimensional vector space into Cartesian products of `pq_factors` low-dimensional vector spaces, which are quantized before IVF index clustering. Each vector is stored in `pq_factors` × `pq_nbits` bits. All this reduces time complexity and space requirements at the cost of accuracy. For more details on indexing method and adjusting `pq_factors` (`m` in Milvus) and `pq_nbits` (`nbits` in Milvus) refer to [Milvus documentation](#)

Milvus monitoring

See the [official documentation](#) ↗ for information on setting up detailed monitoring for Milvus with Prometheus and Grafana.

Health check

It might be desirable to setup a simple health check to make sure the service is actually working. Milvus exposes gRPC as well as simple HTTP interfaces to the clients, we can use them to issue basic health checks.

HTTP

Easiest way to setup Milvus health check is to use HTTP interface that by default runs on `19121` port. You can expose it outside the container with `-p 19121:19121` argument in the `docker run` command.

Then simple `curl -f http://<server_ip>:19121/system/status` can be used for basic Milvus service health checking.

gRPC

To issue health check command directly with gRPC, we can use [Cmd](#) ↗ method with `"status"` as an argument. The Milvus' client libraries expose it with `server_status` methods.

For example in [Python client](#) ↗ it is exposed with `server_status()`:

```
# pip install pymilvus==1.1.2

import milvus
import sys

HOST = 'localhost'
PORT = 19530

cli = milvus.Milvus(HOST, PORT)
# Returns (Status, "OK") on success
if cli.server_status()[1] != "OK":
    sys.exit(1)
```

Docker

For builtin docker health check feature you can use `--health-cmd "curl -f http://localhost:19121/system/status"` argument in the `docker run` command.



This health check mechanism only checks whether Milvus is running or not. It doesn't verify that it has access to its storage/MySQL or that the actual queries will work.

Unfortunately, more sophisticated health checks aren't currently available in Milvus.

Milvus High Availability Setup

See the [tutorial](#) for information on setting up highly available Milvus cluster.

Backups

As mentioned earlier, Identity stores all the embeddings in Milvus, with additional Faces/Identities/Groups metadata stored in MySQL. Milvus itself stores the embeddings on disk and some additional internal metadata in MySQL. Because of

that when making a backup we need to make sure to include the Milvus disk storage and both MySQL databases (Identity's and Milvus').

⚠ To guarantee the consistency of the data ensure no writes are made while making all the backups.

Identity

For MySQL backups any standard tool will do, like [mysqldump](#) ↗. Simple backup of `<IDENTITY_DB_NAME>` should suffice.

Example commands with docker:

- Backup:

```
docker run -it --rm --link <MYSQL_CONTAINER_NAME> -e MYSQL_PWD=
<DB_PASSWORD> mysql:5.7 mysqldump \
    --add-drop-database --column-statistics=0 \
    -h <MYSQL_CONTAINER_NAME> \
    -P 3306 \
    -u <DB_USER> \
    <IDENTITY_DB_NAME> \
    > identity_dump.sql
```

- Restore:

```
docker run -i --rm --link <MYSQL_CONTAINER_NAME> -e MYSQL_PWD=
<DB_PASSWORD> mysql:5.7 mysql \
    -h <MYSQL_CONTAINER_NAME> \
    -P 3306 \
    -u <DB_USER> \
    <IDENTITY_DB_NAME> \
    < identity_dump.sql
```

Milvus

Milvus provides a data migration tool [MilvusDM](#) for exporting/importing data. It can be used as simple backup/restore tool.

```
python3 -m pip install pymilvusdm==1.0.0
```

Note: See also [MilvusDM documentation](#) for more information.

As MilvusDM requires direct access to Milvus database files, you should run flush operation to ensure everything is actually on disk before making a backup.

Example:

HTTP

You can use Milvus' HTTP interface:

```
curl -X PUT \
  -H "accept: application/json" \
  -d "{\"flush\": {\"collection_names\": [\"
<MILVUS_COLLECTION_PREFIX>_faces\"]}}\" \
  "http://<MILVUS_CONTAINER_IP_ADDRESS>:19121/system/task"
```

gRPC

You can use Milvus Python SDK:

```
from milvus import Milvus
milvus = Milvus('<MILVUS_CONTAINER_IP_ADDRESS>', '19530')
milvus.flush(['<MILVUS_COLLECTION_PREFIX>_faces'])
print(milvus.get_collection_stats('<MILVUS_COLLECTION_PREFIX>_faces')[1])
```

You can also just take a snapshot of Milvus storage disk, and regular database backup of `<MILVUS_DB_NAME>` database.

Backup

MilvusDM requires direct access to Milvus' storage so it's the easiest if the volume is mounted from the host machine.

Prepare Milvus-to-HDF5 config file for MilvusDM, `M2H.yaml`:

```
M2H:
  milvus_version: 1.1.1
  source_milvus_path: '<MILVUS_STORAGE_PATH>'
  mysql_parameter:
    host: '<MYSQL_CONTAINER_IP_ADDRESS>'
    user: '<DB_USER>'
    port: <DB_PORT>
    password: '<DB_PASSWORD>'
    database: '<MILVUS_DB_NAME>'
  source_collection:
    <MILVUS_COLLECTION_PREFIX>_faces:
  data_dir: '<BACKUP_DIR>'
```

Perform backup:

```
milvusdm --yaml M2H.yaml
```

Your backup should now reside in `<BACKUP_DIR>/<UUID>`.

Restore

⚠ By default MilvusDM creates import configurations for each partition with `skip` mode which skips existing collections. To be able to insert the embeddings into collection that Identity created, patch resulting import configurations before restoring:

```
sed -i 's|mode: skip|mode: append|g'
<BACKUP_DIR>/<UUID>/ymls/<MILVUS_COLLECTION_PREFIX>_faces/*.yaml
1
```

MilvusDM dumps all the partitions (which in our use case correspond to `model` parameter in Identity API functions) separately so you have to import them one by one:

```
for f in  
<BACKUP_DIR>/<UUID>/ymls/<MILVUS_COLLECTION_PREFIX>_faces/*.yaml; do  
    milvusdm --yaml "$f"  
done
```

Usage

The Identity API container runs a gRPC server that allows you to store, manage and look up identities. The guide below will show you how to use our various [clients](#) to interact with that server.

Initialization

Create a `Client` object.

C#

```
Client(string host, bool ssl = false, string sslCert = null, string  
authCredentials = null, string lbPolicy = null)
```

C++

```
Client Client(string host, bool ssl = false, string ssl_cert = "",  
string auth_credentials = "", string lb_policy = "")
```

Go

```
NewClient(host string, ssl bool, sslCert []byte, authCredentials  
string, lbPolicy string)(*Client, error)
```

Java

```
Client(String host, boolean ssl = false, byte[] sslCert = null,  
String authCredentials = null, String lbPolicy = null)
```

Node

```
Client(host: string, ssl: Boolean = false, sslCert: string = null,  
authCredentials: string = null, lbPolicy: string = null) => Client
```

Python

```
Client(host: str, ssl: bool = False, ssl_cert: Optional[bytes] =  
None, auth_credentials: Optional[str] = None, lb_policy:  
Optional[str] = None) -> Client
```

Ruby

```
Client.new(host, ssl: false, ssl_cert: nil, auth_credentials: nil,  
lb_policy: nil) -> Client
```

Arguments

host: A string, the url where the Identity API is running. For example, if you are running the API in a local Docker container on port `5656` with port forwarding, you should use a host of `localhost:5656`.

ssl: A boolean, describes whether to enable SSL/TLS encryption (requires encryption enabled on the [server](#)).

ssl_cert: A string, certificate to use for validation instead of built in OS Root Certificates.

auth_credentials: A string of the form `username:password`, where `username` and `password` are meant to be used as authentication parameters for basic gRPC authentication. The `username` cannot contain any colons or control characters, and `password` cannot contain any control characters (see [RFC 7617](#) ↗).

lb_policy: A string, name of the policy to be used by gRPC in the client side load balancing.

Return

client: The Identity `Client` object.

Example

C#

```
using Identity;

// Insecure connection
var client = new Identity.Client("localhost:5656");

// With SSL and custom certificate
var sslCert = File.ReadAllText("ssl.cert");
var client = new Identity.Client("localhost:5656", true, sslCert);

// With SSL, custom certificate and basic authentication
var sslCert = File.ReadAllText("ssl.cert");
var client = new Identity.Client("localhost:5656", true, sslCert,
    "username:password");

// With round_robin client side load balancing
var client = new Identity.Client("dns:///<dns resolvable
name>:5656", false, null, null, "round_robin");
```

C++

```
#include <paravision/identity.h>

// Insecure connection
paravision::identity::Client client("localhost:5656");

// With SSL and custom certificate
string ssl_cert;
{
    auto ss = ostringstream{};
    ifstream file("ssl.cert", ifstream::in | ifstream::binary);
    ss << file.rdbuf();
    ssl_cert = ss.str();
}
paravision::identity::Client client("localhost:5656", true,
ssl_cert);

// With SSL, custom certificate and basic authentication
string ssl_cert;
{
    auto ss = ostringstream{};
    ifstream file("ssl.cert", ifstream::in | ifstream::binary);
    ss << file.rdbuf();
    ssl_cert = ss.str();
}
paravision::identity::Client client("localhost:5656", true,
ssl_cert, "username:password");

// With round_robin client side load balancing
paravision::identity::Client client("dns:///<dns resolvable
name>:5656", false, "", "", "round_robin");
```

Go

```
import {
    "io/ioutil"

    identity "golang.paravision.ai/identity-go-client/v6"
    pb "golang.paravision.ai/identity-go-client/v6/proto"
}

// Insecure connection
client, err := identity.NewClient("localhost:5656", false, nil, "",
    "")

// With SSL and custom certificate
sslCert, err := ioutil.ReadFile("ssl.cert")
if err != nil {
    return nil, err
}
client, err := identity.NewClient("localhost:5656", true, sslCert,
    "", "")

// With SSL, custom certificate and basic authentication
sslCert, err := ioutil.ReadFile("ssl.cert")
if err != nil {
    return nil, err
}
client, err := identity.NewClient("localhost:5656", true, sslCert,
    "username:password", "")

// With round_robin client side load balancing
client, err := identity.NewClient("dns:///<dns resolvable
name>:5656", false, nil, nil, "round_robin")
```

Java

```
import ai.paravision.identity.*;

// Insecure connection
var client = new Client("localhost:5656");

// With SSL and custom certificate
var sslCert = Files.readAllBytes("ssl.cert")
var client = new Client("localhost:5656", true, sslCert);

// With SSL, custom certificate and basic authentication
sslCert = Files.readAllBytes("ssl.cert")
var client = new Client("localhost:5656", true, sslCert,
"username:password");

// With round_robin client side load balancing
var client = new Client("dns:///<dns resolvable name>:5656", false,
null, null, "round_robin");
```

Node

```
const { Client } = require('@paravision/identity-js-client')

// Insecure connection
const client = Client("localhost:5656");

// With SSL and custom certificate
const sslCert = fs.readFileSync("ssl.cert");
const client = new Client("localhost:5656", true, sslCert);

// With SSL, custom certificate and basic authentication
const sslCert = fs.readFileSync("ssl.cert");
const client = new Client("localhost:5656", true, sslCert,
"username:password");

// With round_robin client side load balancing
const client = new Client("dns:///<dns resolvable name>:5656",
false, null, null, "round_robin");
```

Python

```
import paravision_identity as identity

# Insecure connection
client = identity.Client("localhost:5656")

# With SSL and custom certificate
with open("ssl.cert", "rb") as certfile:
    ssl_cert = certfile.read()
client = identity.Client("localhost:5656", ssl=True,
ssl_cert=ssl_cert)

# With SSL, custom certificate and basic authentication
with open("ssl.cert", "rb") as certfile:
    ssl_cert = certfile.read()
client = identity.Client("localhost:5656", ssl=True,
ssl_cert=ssl_cert, auth_credentials="username:password")

# With round_robin client side load balancing
client = identity.Client("dns:///<dns resolvable name>:5656",
lb_policy="round_robin")
```

Ruby

```
require 'paravision-identity-client'
include Paravision

# Insecure connection
client = Identity::Client.new('localhost:5656')

# With SSL and custom certificate
ssl_cert = File.binread('ssl.cert')
client = Identity::Client.new('localhost:5656', ssl: true, ssl_cert:
ssl_cert)

# With SSL, custom certificate and basic authentication
ssl_cert = File.binread('ssl.cert')
client = Identity::Client.new('localhost:5656', ssl: true, ssl_cert:
ssl_cert, auth_credentials: 'username:password')

# With round_robin client side load balancing
client = Identity::Client.new('dns:///<dns resolvable name>:5656',
lb_policy: "round_robin")
```

 Refer to the [tutorial](#) for more details on SSL connection set-up.

 **Note:** In some implementations before you can start adding identities, you must first use the [Connect](#) function below to connect to the server.

 gRPC supports client-side load balancing. To activate it in our clients, you need to:

- provide a correct value for the `lb_policy` constructor argument. The full list of policies supported by gRPC is available in its [load balancing documentation](#).
- provide the host address as a full URI with dns scheme (`dns:///<dns_resolvable_name>`). This is because internally gRPC is using DNS to resolve the list of available backend addresses.

You can also set the `PV_CONNECTION_AGE_MS` [environment variable](#) on the server to make your clients reconnect to the server periodically, allowing them to re-resolve backend addresses. This is useful when your server deployment scales up, as clients will be aware of the changes faster and will be able to accommodate the changes into load balancing strategy sooner.

Connect

Connect connects your client to the Identity API's gRPC server.

C#

```
void Connect()
```

C++

```
paravision::Status Connect()
```

Java

```
void connect() throws ClientException
```

Node

```
async connect() => void
```

Python

```
connect() -> None
```

Ruby

```
connect() -> nil
```

Example

C#

```
client.Connect();
```

C++

```
auto status = client.Connect();  
if(!status.ok())  
{  
    cerr << status.error_message << endl;  
    exit(1);  
}
```

Java

```
client.connect();
```

Node

```
await client.connect();
```

Python

```
client.connect()
```

Ruby

```
client.connect
```

Close

Close the connection to the Identity API's gRPC server.

C#

```
void Close()
```

C++

```
void Close()
```

Go`Close()`**Java**`void close()`**Node**`close() => void`**Python**`close() -> None`**Ruby**`close() -> nil`

Example

C#`client.Close();`**C++**

```
client.Close();
```

Go

```
client.Close()
```

Java

```
client.close();
```

Node

```
client.close();
```

Python

```
client.close()
```

Ruby

```
client.close
```

Lookup

Lookup finds identities that closely match each of the given embeddings.

C#

```
List<List<pb.IdentityMatch>> Lookup(
    IEnumerable<IEnumerable<float>> embeddings,
    IEnumerable<string> groupIds = null,
    int limit = 1,
    string model = "",
    float scalingFactor = 2.0f,
    int bucketsLimit = 32)
```

```
async Task<List<List<pb.IdentityMatch>>> LookupAsync(
    IEnumerable<IEnumerable<float>> embeddings,
    IEnumerable<string> groupIds = null,
    int limit = 1,
    string model = "",
    float scalingFactor = 2.0f,
    int bucketsLimit = 32)
```

C++

```
paravision::Status Lookup(
    vector<vector<IdentityMatch>> &matches,
    const vector<Embedding> &embeddings,
    const vector<string> &group_ids = {},
    const int limit = 1,
    const string &model = "",
    const float scaling_factor = 2.0f,
    const int buckets_limit = 32)
```

Go

```
Lookup(embeddings [][]float32, groupIds []string, limit int32, model
string, scalingFactor float32, bucketsLimit int)
([][]*pb.IdentityMatch, error)
```

Java

```
List<LookupIdentity> lookup(
List<List<Float>> embeddings,
List<String> groupIds = [],
int limit = 1,
String model = "",
float scalingFactor = 2,
int bucketsLimit = 32
) throws ClientException
```

Node

```
async lookup(
embeddings: Array<(!Embedding|Array<float>)>,
groupIds: Array<string> = [],
limit: int = 1,
model: string="",
scalingFactor: float = 2.0,
bucketsLimit: int = 32) => Array<!LookupIdentity>
```

Python

```
lookup(embeddings: List[Embedding], group_ids: List[str] = [],
limit: int = 1,
model: str = "",
scaling_factor: float = 2.0, buckets_limit: int = 32) ->
List[List[Tuple[Identity, float]]]
```

Ruby

```
lookup(embeddings, group_ids: [], limit: 1, model: '',
scaling_factor: 2.0, buckets_limit: 32)
-> Array[Array[IdentityMatch]]
```

Arguments

embeddings: A list of [embeddings](#) to lookup. These embeddings can be generated using our [Recognition SDK ↗](#) and our [Processor API ↗](#).

group_ids: A list of group ids. The identity search will be limited to identities belonging to these groups.

limit: An integer, the maximum number of identities returned per embedding. Default is 1.

model: A string that represents the model used to generate the embeddings in the argument. The identity search will be limited to identities that have associated embeddings generated using this model. This argument can be left blank/empty in which case the search will be limited to identities that have associated embeddings with no model name attached.

scaling_factor: A float greater than 1, optional. Rate at which number of considered embeddings is increased while searching for requested number of identities.

buckets_limit: An integer, optional. Corresponds to `nprobe` parameter from [Milvus ↗](#), used while searching for requested number of identities.

 **scaling_factor, buckets_limit** are only used when Milvus is configured as a database.

Return

matches: A 2D matrix of [IdentityMatches](#), where `matches[i]` is a list of up to `limit` identity/score pairs for the *i*th embedding. There might be fewer than `limit` elements if the database has fewer than `limit` identities. The [score](#) represents how similar the embedding is to the closest one belonging to the identity.

status: C++, [Status](#) object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
float[] embedding1;
float[] embedding2;
...
var matches = client.Lookup(
    new List<float[]>{embedding1, embedding2},
    new string[]{"group1_id"},
    5);
// matches for first identity are available in matches[0]
// and for the second in matches[1]
```

C++

```
vector<float> embedding1;
vector<float> embedding2;
...
vector<vector<IdentityMatch>>> matches;
auto status = client.Lookup(matches, {embedding1, embedding2},
                             {"group1_id"}, 5);

if(!status.ok())
{
    cerr << "Lookup failed! " << status.error_message << endl;
    exit(1);
}

// matches for first identity are available in matches[0]
// and for the second in matches[1]
```

Go

```

embedding1 []float32
embedding2 []float32
...
matches, err := client.Lookup([][]float32{embedding1, embedding2},
                                []string{"group1_id"}, 5,
                                "tarka")
if err != nil {
    log.Fatalf("Error looking up identity: %v", err)
}

// matches for first identity are available in matches[0]
// and for the second in matches[1]

```

Java

```

List<Float> embedding1 = ...;
List<Float> embedding2 = ...;
...
var matches = client.lookup(
    Arrays.asList(embedding1, embedding2),
    Arrays.asList("group1_id"),
    5);
// matches for first identity are available in
matches.get(0).getMatchList()
// and for the second in matches.get(1).getMatchList()

```

Node

```

const embedding1 = [...];
const embedding2 = [...];
var matches = await client.lookup(
    [embedding1, embedding2], ["group1_id"], 5, "tarka");

// matches for first identity are available in
matches[0].getMatchesList()
// and for the second in matches[1].getMatchesList()

```

Python

```
embedding1: List[float] = []
embedding2: List[float] = []
...
matches = client.lookup(
    [embedding1, embedding2], ["group1_id"], 5)

# matches for first identity are available in matches[0]
# and for the second in matches[1]
```

Ruby

```
embedding1 = []
embedding2 = []
...
matches = client.lookup(
    [embedding1, embedding2], group_ids: ['group1_id'], limit: 5
)

# matches for first identity are available in matches[0]
# and for the second in matches[1]
```

Create Identity

Create Identity creates a new identity.

C#

```
pb.Identity CreateIdentity(
    IEnumerable<float> embedding,
    float threshold,
    float quality,
    IEnumerable<string> groupIds = null,
    string model = "",
    string externalId = "",
    float scalingFactor = 2.0f,
    int bucketsLimit = 32,
    IEnumerable<CreateIdentityOptions> options = null)
```

```

async Task<pb.Identity> CreateIdentityAsync(
    IEnumerable<float> embedding,
    float threshold,
    float quality,
    IEnumerable<string> groupIds = null,
    string model = "",
    string externalId = "",
    float scalingFactor = 2.0f,
    int bucketsLimit = 32,
    IEnumerable<CreateIdentityOptions> options = null)

```

C++

```

paravision::Status CreateIdentity(
    Identity &identity,
    const Embedding &embedding,
    const float threshold,
    const float quality,
    const vector<string> &group_ids = {},
    const string &model = "",
    const string &external_id = "",
    const float scaling_factor = 2.0f,
    const int buckets_limit = 32,
    const std::vector<CreateIdentityOptions> &options = {})

```

Go

```

CreateIdentity(embedding []float32, threshold float32,
    quality float32, groupIds []string, model string, externalId string,
    scalingFactor float32, bucketsLimit int, options
    []pb.CreateIdentitiesRequest_Options)
(*pb.Identity, error)

```

Java

```
Identity createIdentity(
    List<Float> embedding,
    float threshold,
    float quality,
    List<String> groupIds = [],
    String model = "",
    String externalId = "",
    float scalingFactor = 2,
    int bucketsLimit = 32,
    Iterable<CreateIdentitiesRequest.Options> options
) throws ClientException
```

Node

```
async createIdentity(
    embedding: Array<float>,
    threshold: float,
    quality: float,
    groupIds: Array<string> = [],
    model: string = "",
    external_id: string = "",
    scalingFactor: float = 2.0,
    bucketsLimit: int = 32,
    options: Array<!CreateIdentityOptions> = []) => !Identity
```

Python

```
create_identity(embedding: Embedding, threshold: float,
    quality: float, group_ids: List[str] = [], model: str = "",
    external_id: str = "",
    scaling_factor: float = 2.0, buckets_limit: int = 32, options:
    Iterable[CreateIdentityOptions] = [])
-> Identity
```

Ruby

```
create_identity(embedding, threshold,  
quality, group_ids: [], model: '', external_id: nil,  
scaling_factor: 2.0, buckets_limit: 32, options: [])  
-> Identity
```

Arguments

embedding: An [embedding](#) of the face belonging to the new identity. This embedding can be generated using our [Recognition SDK ↗](#) and our [Processor API ↗](#).

threshold: A float representing a valid [score](#) value. If nonzero, the operation will not succeed (an error will be returned) if the given embedding matches an already-existing embedding in the database with a score greater than this threshold.

quality: A float between 0 and 1, the quality score of the embedding.

group_ids: A list, containing the IDs of any groups that the new identity belongs to.

model: A string that represents the model used to generate the embedding. This argument can be left blank if the user does not wish to specify a model name.

external_id: String, optional. Any custom value to be stored alongside this identity. Can optionally be used when searching for an identity.

scaling_factor: A float greater than 1, optional. Rate at which number of considered embeddings is increased while searching for already-existing embedding for **threshold** check.

buckets_limit: An integer, optional. Corresponds to `nprobe` parameter from [Milvus ↗](#), used while searching for already-existing embedding for **threshold** check.

options: A list of [CreateIdentityOptions](#), additional options allowing change of default behavior.

 **scaling_factor** and **buckets_limit** are only used when Milvus is configured as a database.

Return

identity: An `Identity`, the newly created identity.

status: C++, `Status` object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var identity = client.CreateIdentity(  
    embedding, 0.5, 0.9, new string[]{"group1_id", "group2_id"},  
    "tarka");
```

C++

```
Identity identity;  
auto status = client.CreateIdentity(identity, embedding, 0.5f, 0.9f,  
    {"group1_id", "group2_id"},  
    "tarka");  
  
if(!status.ok())  
{  
    cerr << "Create Identity failed! " << status.error_message <<  
    endl;  
    exit(1);  
}
```

Go

```
identity, err := client.CreateIdentity(  
    embedding, 0.5, 0.9, []string{"group1_id", "group2_id"},  
    "tarka")  
if err != nil {  
    log.Fatalf("Error creating identity: %v", err)  
}
```

Java

```
Identity identity = client.createIdentity(  
    embedding, 0.5, 0.9,  
    Arrays.asList("group1_id", "group2_id"),  
    "tarka");
```

Node

```
var identity = await client.CreateIdentity(  
    embedding,  
    0.5,  
    (0.9)[("group1_id", "group2_id")],  
    "tarka"  
);
```

Python

```
identity = client.create_identity(embedding, 0.5, 0.9,  
    ["group1_id", "group2_id"],  
    "tarka")
```

Ruby

```
identity = client.create_identity(  
    embedding, 0.5, 0.9, group_ids: ['group1_id', 'group2_id'], model:  
    'tarka'  
)
```

Create Identities

Create Identities creates a set of new identities.

C#

```
List<pb.Identity> CreateIdentities(  
    IEnumerable<IEnumerable<float>> embeddings,  
    float threshold,  
    IEnumerable<float> qualities,  
    IEnumerable<string> groupIds = null,  
    string model = "",  
    IEnumerable<string> externalIds = null,  
    float scalingFactor = 2.0f,  
    int bucketsLimit = 32,  
    IEnumerable<CreateIdentitiesOptions> options = null)
```

```
async Task<List<pb.Identity>> CreateIdentitiesAsync(  
    IEnumerable<IEnumerable<float>> embeddings,  
    float threshold,  
    IEnumerable<float> qualities,  
    IEnumerable<string> groupIds = null,  
    string model = "",  
    IEnumerable<string> externalIds = null,  
    float scalingFactor = 2.0f,  
    int bucketsLimit = 32,  
    IEnumerable<CreateIdentitiesOptions> options = null)
```

C++

```

paravision::Status CreateIdentities(
    vector<Identity> &identities,
    const vector<Embedding> &embeddings,
    const float threshold,
    const vector<float> &qualities,
    const vector<string> &group_ids = {},
    const string &model = "",
    const vector<string> &external_ids = {},
    const float scaling_factor = 2.0f,
    const int buckets_limit = 32,
    const std::vector<CreateIdentitiesOptions> &options = {})

```

Go

```

CreateIdentities(embeddings [][]float32, threshold float32,
    qualities []float32, groupIds []string, model string, externalIds
    []string,
    scalingFactor float32, bucketsLimit int, options
    []pb.CreateIdentitiesRequest_Options)
    ([]*pb.Identity, error)

```

Java

```

List<Identity> createIdentities(
    List<List<Float>> embeddings,
    float threshold,
    List<Float> qualities,
    List<String> groupIds = [],
    String model = "",
    List<String> externalIds = [],
    float scalingFactor = 2,
    int bucketsLimit = 32,
    Iterable<CreateIdentitiesRequest.Options> options
    ) throws ClientException

```

Node

```

async createIdentities(
  embeddings: Array<!Embedding|Array<float>>>,
  threshold: float,
  qualities: Array<float>,
  groupIds: Array<string> = [],
  model: string = "",
  externalIds: Array<string> = [],
  scalingFactor: float = 2.0,
  bucketsLimit: int = 32,
  options: Array<!CreateIdentitiesOptions> = []) => !Identity

```

Python

```

create_identities(embeddings: List[Embedding], threshold: float,
  qualities: List[float], group_ids: List[str] = [],
  model: str = "", external_ids: List[str] = [],
  scaling_factor: float = 2.0, buckets_limit: int = 32, options:
  Iterable[CreateIdentityOptions] = [])

```

Ruby

```

create_identities(embeddings, threshold, qualities,
  group_ids: [], model: '', external_ids: [],
  scaling_factor: 2.0, buckets_limit: 32, options: []) ->
  Array[Identity]

```

Arguments

embeddings: A list of [embeddings](#), each one corresponding to the face of a new identity. These embeddings can be generated using our [Recognition SDK ↗](#) and our [Processor API ↗](#).

threshold: A float representing a valid [score](#) value. If nonzero, the operation will not succeed (an error will be returned) if at least one of the given embeddings matches

an already-existing embedding in the database with a score greater than this threshold.

qualities: A list of floats between 0 and 1, the quality scores of the embeddings.

group_ids: A list, containing the IDs of any groups that the new identities belong to.

model: A string that represents the models used to generate the embeddings. This argument can be left blank if the user does not wish to specify a model name.

external_ids: List of strings, optional. One per every identity created. Any custom value to be stored alongside each corresponding identity. Can optionally be used when searching for an identity.

scaling_factor: A float greater than 1, optional. Rate at which number of considered embeddings is increased while searching for already-existing embedding for **threshold** check.

buckets_limit: An integer, optional. Corresponds to `nprobe` parameter from [Milvus](#) ↗, used while searching for already-existing embedding for **threshold** check.

options: A list of `CreateIdentitiesOptions`, additional options allowing change of default behavior.

 **scaling_factor** and **buckets_limit** are only used when Milvus is configured as a database.

Return

identities: List of `Identities`, the new identities.

status: C++, `Status` object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var identities = client.CreateIdentities(  
    embeddings, 0.5, qualities, new string[]{"group1_id"}, "tarka");
```

C++

```
vector<Identity> identities;  
auto status = client.CreateIdentities(  
    identities, embeddings, 0.5, qualities, {"group1_id"}, "tarka");  
if(!status.ok())  
{  
    cerr << "Error creating identities! " << status.error_message <<  
endl;  
    exit(1);  
}
```

Go

```
identities, err := client.CreateIdentities(  
    embeddings, 0.5, qualities, []string{"group1_id"}, "tarka")  
if err != nil {  
    log.Fatalf("Error creating identities: %v", err)  
}
```

Java

```
List<Identities> identities = client.createIdentities(  
    embeddings, 0.5, qualities,  
    Arrays.asList("group1_id"), "tarka");
```

Node

```
var identities = await client.CreateIdentities(  
    embeddings,  
    0.5,  
    qualities["group1_id"],  
    "tarka"  
);
```

Python

```
identities = client.create_identities(  
    embeddings, 0.5, qualities, ["group1_id"], "tarka")
```

Ruby

```
identities = client.create_identities(  
    embeddings, 0.5, qualities, group_ids: ['group1_id'], model:  
    'tarka'  
)
```

Get Identity

Get identity retrieves an identity.

C#

```
pb.Identity GetIdentity(string id, string externalId)
```

```
async Task<pb.Identity> GetIdentityAsync(string id, string  
externalId)
```

C++

```
paravision::Status GetIdentity(Identity &identity, const string &id,  
const string &external_id)
```

Go

```
GetIdentity(id string, externalId string) (*pb.Identity, error)
```

Java

```
Identity getIdentity(String id, String externalId) throws  
ClientException
```

Node

```
async getIdentity(id: string, externalId: string) => !Identity
```

Python

```
get_identity(id: str, external_id: str) -> Identity
```

Ruby

```
get_identity(id: nil, external_id: nil) -> Identity
```

Arguments

id: A string, the identity ID.

external_id: A string, optional. If provided, `id` becomes optional, and only an identity with a matching `external_id` will be returned.

If both `id` and `external_id` are used, this function will search for an identity that matches both criteria.

Return

identity: An `Identity`, the requested identity.

status: C++, `Status` object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var identity = client.GetIdentity("identity_id");
```

C++

```
Identity identity;  
auto status = client.GetIdentity(identity, "identity_id");  
if(!status.ok())  
{  
    cerr << "Error getting identity! " << status.error_message <<  
endl;  
    exit(1);  
}
```

Go

```
identity, err := client.GetIdentity("identity_id")
if err != nil {
    log.Fatalf("Error getting identity: %v", err)
}
```

Java

```
Identity identity = client.getIdentity("identity_id");
```

Node

```
var identity = await client.getIdentity("identity_id");
```

Python

```
identity = client.get_identity("identity_id")
```

Ruby

```
identity = client.get_identity(id: 'identity_id')
```

Get Identities

Get identities retrieves a set of identities.

C#

```
Tuple<List<pb.Identity>, string, int> GetIdentities(
    IEnumerable<string> groupIds = null,
    int pageSize = 100,
    string pageToken = "")
```

```
async Task<Tuple<List<pb.Identity>, string, int>>
GetIdentitiesAsync(
    IEnumerable<string> groupIds = null,
    int pageSize = 100,
    string pageToken = "")
```

C++

```
paravision::Status GetIdentities(tuple<vector<Identity>, string,
int> &result,
const vector<string> &group_ids = {},
const int page_size = 100,
const string &page_token = "")
```

Go

```
GetIdentities(groupIds []string, pageSize int, pageToken string)
([]*pb.Identity, string, int, error)
```

Java

```
PaginatedResult<Identity> getIdentities(
    List<String> groupIds = [],
    int pageSize = 100,
    String pageToken = ""
    ) throws ClientException
```

Node

```
async getIdentities(  
  groupIds: Array<string> = [],  
  pageSize: int = 100,  
  pageToken: string = "") => [Array<!Identity>, string, int]
```

Python

```
get_identities(group_ids: List[str] = [], page_size: int = 100,  
page_token: str = "") -> Tuple[List[Identity], str, int]
```

Ruby

```
get_identities(group_ids: [], page_size: 100, page_token: '')  
-> [Array[Identity], String, Integer]
```

Arguments

group_ids: A list of group ids. The identity search will be limited to identities belonging to these groups. If this is empty, identities will be searched across all groups. Default is an empty list.

page_size: An integer, the number of results to return. Defaults to 100.

page_token: A string specifying the page of results to return. Using the default empty string will return the first page of results. Subsequent calls should use the returned `next_page_token` for the value of this parameter.

Return

identities: List of `Identities`, the retrieved identities.

next_page_token: A string, the value of the page token associated with the next page of results. If this is an empty string, there are no more results.

total_size: An integer, the total count of identities that belong to the groups.

status: C++, [Status](#) object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var result = client.GetIdentities(new string[]{"group1_id"}, 5, "");  
// result.Item1 is list of identities  
// result.Item2 is next page token  
// result.Item3 is total size
```

C++

```
tuple<vector<Identity>, string, int> result;  
auto status = client.GetIdentities(result, {"group1_id"}, 5, "");  
if(!status.ok())  
{  
    cerr << "Error getting identities! " << status.error_message <<  
endl;  
    exit(1);  
}
```

Go

```
identities, next_page_token, total_size, err :=  
client.GetIdentities(  
    []string{"group1_id"}, 5, "")  
if err != nil {  
    log.Fatalf("Error getting identities: %v", err)  
}
```

Java

```
PaginatedResult<Identity> result =  
client.getIdentities(Arrays.asList("group1_id"), 5, "");  
// result.value is a list of identities  
// result.nextToken is next page token  
// result.totalSize is total size
```

Node

```
const [identities, nextPageToken, totalSize] = await  
client.getIdentities(  
  ["group1_id"],  
  5,  
  ""  
);
```

Python

```
identities, next_page_token, total_size = client.get_identities(  
    ["group1_id"],  
    5,  
    ""  
)
```

Ruby

```
identities, next_page_token, total_size = client.get_identities(  
  group_ids: ['group1_id'], page_size: 5, page_token: ''  
)
```

 For a tutorial on pagination, see [here](#).

Delete Identity

Delete identity deletes an identity.

C#

```
int DeleteIdentity(string id, string externalId)
```

```
async Task<int> DeleteIdentityAsync(string id, string externalId)
```

C++

```
paravision::Status DeleteIdentity(int &rows_affected, const string  
&id, const string &external_id)
```

Go

```
DeleteIdentity(id string, externalId string) (int, error)
```

Java

```
int deleteIdentity(String id, String externalId) throws  
ClientException
```

Node

```
async deleteIdentity(id: string, externalId: string) => int
```

Python

```
delete_identity(id: Optional[str], external_id: Optional[str]) ->
int
```

Ruby

```
delete_identity(id, external_id) -> Integer
```

Arguments

id: A string, optional, the ID of the identity to delete.

external_id: A string, optional, the external ID of the identity to delete.

If both `id` and `external_id` are used, this function will delete an identity that matches both criteria. If one of the arguments is set to empty string, an identity matching the other argument will be deleted.

Return

rows_affected: The number of identities removed as a result of this request.

status: C++, `Status` object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var rowsAffected = client.DeleteIdentity("identity_id");
```

C++

```
int rows_affected;
auto status = client.DeleteIdentity(rows_affected, "identity_id");
if(!status.ok())
{
    cerr << "Error deleting identity! " << status.error_message <<
endl;
    exit(1);
}
```

Go

```
rowsAffected, err := client.DeleteIdentity("identity_id")
if err != nil {
    log.Fatalf("Error deleting identity: %v", err)
}
```

Java

```
int rowsAffected = client.deleteIdentity("identity_id");
```

Node

```
var rowsAffected = await client.deleteIdentity("identity_id");
```

Python

```
rows_affected = client.delete_identity("identity_id")
```

Ruby

```
rows_affected = client.delete_identity('identity_id')
```

Delete Identities

Delete identities deletes a set of identities.

C#

```
int DeleteIdentities(IEnumerable<string> ids, IEnumerable<string> externalIds)
```

```
async Task<int> DeleteIdentitiesAsync(IEnumerable<string> ids, IEnumerable<string> externalIds)
```

C++

```
paravision::Status DeleteIdentities(int &rows_affected, const vector<string> &ids, const vector<string> &external_ids)
```

Go

```
DeleteIdentities(ids []string, externalIds []string) (int, error)
```

Java

```
int deleteIdentities(List<String> ids, List<String> externalIds)  
throws ClientException
```

Node

```
async deleteIdentities(ids: Array<string>, externalIds:  
Array<string>) => int
```

Python

```
delete_identities(ids: Optional[List[str]], external_ids:  
Optional[List[str]]) -> int
```

Ruby

```
delete_identities(ids, external_ids) -> Integer
```

Arguments

ids: A list of strings, optional, the IDs of the identities to delete.

external_ids: A list of strings, optional, the external IDs of the identities to delete.

If both `ids` and `external_ids` are used, this function will delete identities that match both criteria, pairwise. If one of the arguments is set to empty list, identities matching the other argument will be deleted.

Return

rows_affected: The number of identities removed as a result of this request.

status: C++, `Status` object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var rowsAffected = client.DeleteIdentities(new List<string>
{"identity1_id", "identity2_id"});
```

C++

```
int rows_affected;
auto status = client.DeleteIdentities(rows_affected,
{"identity1_id", "identity2_id"});
if(!status.ok())
{
    cerr << "Error deleting identities! " << status.error_message <<
endl;
    exit(1);
}
```

Go

```
rowsAffected, err :=
client.DeleteIdentities([]string{"identity1_id", "identity2_id"})
if err != nil {
    log.Fatalf("Error deleting identities: %v", err)
}
```

Java

```
int rowsAffected =
client.deleteIdentities(Arrays.asList("identity1_id",
"identity2_id"));
```

Node

```
var rowsAffected = await client.deleteIdentities(["identity1_id"]);
```

Python

```
rows_affected = client.delete_identities(["identity1_id",  
"identity2_id"])
```

Ruby

```
rows_affected = client.delete_identities(['identity1_id',  
'identity2_id'])
```

Update Identity Groups

Update Identity Groups adds and removes an identity from specified groups.

C#

```
pb.Identity UpdateIdentityGroups(string id,  
    IEnumerable<string> addGroupIds = null,  
    IEnumerable<string> removeGroupIds = null,  
    bool flush = true)
```

```
async Task<pb.Identity> UpdateIdentityGroupsAsync(string id,  
    IEnumerable<string> addGroupIds = null,  
    IEnumerable<string> removeGroupIds = null,  
    bool flush = true)
```

C++

```
paravision::Status UpdateIdentityGroups(Identity &identity,  
    const string &id,  
    const vector<string> &add_group_ids,  
    const vector<string> &remove_group_ids,  
    bool flush = true)
```

Go

```
UpdateIdentityGroups(id string, addGroupIds []string, removeGroupIds []string, flush bool) (*pb.Identity, error)
```

Java

```
Identity updateIdentityGroups(  
    String id,  
    List<String> addGroupIds = [],  
    List<String> removeGroupIds = [],  
    boolean flush = true  
    ) throws ClientException
```

Node

```
async updateIdentityGroups(  
    identityId: string,  
    addGroupIds: Array<string> = [],  
    removeGroupIds: Array<string> = [],  
    flush: Boolean = true) => !Identity
```

Python

```
update_identity_groups(id: str, add_group_ids: List[str],  
    remove_group_ids: List[str], flush: bool = True) -> Identity
```

Ruby

```
update_identity_groups(id, add_group_ids, remove_group_ids, flush:  
    true) -> Identity
```

Arguments

id: A string, the ID of the identity to update.

add_group_ids: A list of strings, the ids of the groups to add the identity to.

remove_group_ids: A list of strings, the ids of the groups to remove the identity from.

flush: A bool, optional. When true, identity will be immediately available in search operations with new groups at the cost of slowing down this update operation. Defaults to true.

 **flush** is only used when Milvus is configured as a database.

Return

identity: An `Identity`, the updated identity.

status: C++, `Status` object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var identity = client.UpdateIdentityGroups(  
    "identity_id",  
    new List<string>{"admin_id"},  
    new List<string>{"engineers_id"});
```

C++

```
Identity identity;
auto status = client.UpdateIdentityGroups(
    identity, "identity_id", {"admin_id"}, {"engineers_id"});
if(!status.ok())
{
    cerr << "Error updating identity! " << status.error_message <<
endl;
    exit(1);
}
```

Go

```
identity, err := client.UpdateIdentityGroups(
    "identity_id", []string{"admin_id"}, []string{"engineers_id"});
if err != nil {
    log.Fatalf("Error updating identity: %v", err)
}
```

Java

```
Identity identity = client.updateIdentityGroups(
    "identity_id",
    Arrays.asList("admin_id"),
    Arrays.asList("engineers_id"));
```

Node

```
var identity = await client.updateIdentityGroups(
    "identity_id",
    ["admin_id"],
    ["engineers_id"]
);
```

Python

```
identity = client.update_identity_groups(  
    "identity_id", ["admin_id"], ["engineers_id"])
```

Ruby

```
identity = client.update_identity_groups(  
    'identity_id', ['admin_id'], ['engineers_id']  
)
```

Create Group

Create Group creates a new group.

C#

```
pb.Group CreateGroup(string name)
```

```
async Task<pb.Group> CreateGroupAsync(string name)
```

C++

```
paravision::Status CreateGroup(Group &group, const string &name)
```

Go

```
CreateGroup(name string) (*pb.Group, error)
```

Java

```
Group createGroup(String name) throws ClientException
```

Node

```
async createGroup(name: string) => !Group
```

Python

```
create_group(name: str) -> Group
```

Ruby

```
create_group(name) -> Group
```

Arguments

name: A string, the name of the new group.

Return

group: A [Group](#), the new group.

status: C++, [Status](#) object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var group = client.CreateGroup("admin");
```

C++

```
Group group;  
auto status = client.CreateGroup(group, "admin");  
if(!status.ok())  
{  
    cerr << "Error creating group! " << status.error_message <<  
endl;  
    exit(1);  
}
```

Go

```
group, err := client.CreateGroup("admin")  
if err != nil {  
    log.Fatalf("Error creating group: %v", err)  
}
```

Java

```
Group group = client.createGroup("admin");
```

Node

```
var group = await client.createGroup("admin");
```

Python

```
group = client.create_group("admin")
```

Ruby

```
group = client.create_group('admin')
```

Get Group

Get Group retrieves a group.

C#

```
pb.Group GetGroup(string id)
```

```
async Task<pb.Group> GetGroupAsync(string id)
```

C++

```
paravision::Status GetGroup(Group &group, const string &id)
```

Go

```
GetGroup(id string) (*pb.Group, error)
```

Java

```
Group getGroup(String id) throws ClientException
```

Node

```
async getGroup(id: string) => !Group
```

Python

```
get_group(id: str) -> Group
```

Ruby

```
get_group(id) -> Group
```

Arguments

id: A string, the group ID.

Return

group: A [Group](#), the group.

status: C++, [Status](#) object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var group = client.GetGroup("group_id");
```

C++

```
Group group;
auto status = client.GetGroup(group, "group_id");
if(!status.ok())
{
    cerr << "Error getting group! " << status.error_message << endl;
    exit(1);
}
```

Go

```
group, err := client.GetGroup("group_id")
if err != nil {
    log.Fatalf("Error getting group: %v", err)
}
```

Java

```
Group group = client.getGroup("group_id");
```

Node

```
var group = await client.getGroup("group_id");
```

Python

```
group = client.get_group("group_id")
```

Ruby

```
group = client.get_group('group_id')
```

Get Groups

Get groups retrieves a set of groups.

C#

```
Tuple<List<pb.Group>, string, int> GetGroups(IEnumerable<string> ids  
= null,  
    int pageSize = 100,  
    string pageToken = "")
```

```
async Task<Tuple<List<pb.Group>, string, int>>  
GetGroupsAsync(IEnumerable<string> ids = null,  
    int pageSize = 100,  
    string pageToken = "")
```

C++

```
paravision::Status GetGroups(tuple<vector<Group>, string, int>  
&result,  
const vector<string> &ids = {},  
const int page_size = 100,  
const string &page_token = "")
```

Go

```
GetGroups(groupIds []string, pageSize int, pageToken string)  
([]*pb.Group, string, int, error)
```

Java

```
PaginatedResult<Group> getGroups(
    List<String> ids = [],
    int pageSize = 100,
    String pageToken = ""
) throws ClientException
```

Node

```
async getGroups(
    ids: Array<string> = [],
    pageSize: int = 100,
    pageToken: string = "") => [Array<!Group>, string, int]
```

Python

```
get_groups(ids: List[str] = [], page_size: int = 100, page_token:
str = "")
-> Tuple[List[Group], str, int]
```

Ruby

```
get_groups(ids: [], page_size: 100, page_token: '')
-> [Array[Group], String, Integer]
```

Arguments

ids: A list of group ids. If this list is empty or `None`, all groups will be searched. Default is an empty list.

page_size: An integer, the number of results to return. Defaults to 100.

page_token: A string specifying the page of results to return. Using the default empty string will return the first page of results. Subsequent calls should use the

returned `next_page_token` for the value of this parameter.

Return

groups: List of `Groups`, the retrieved groups.

next_page_token: A string, the value of the page token associated with the next page of results. If this is an empty string, there are no more results.

total_size: An integer, the total count of groups.

status: C++, `Status` object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var result = client.GetGroups(new string[]{"group1_id"}, 5, "");  
// result.Item1 is list of groups  
// result.Item2 is next page token  
// result.Item3 is total size
```

C++

```
tuple<vector<Group>, string, int> result;  
auto status = client.GetGroups(result, {"group1_id"}, 5, "");  
if(!status.ok())  
{  
    cerr << "Error getting groups! " << status.error_message <<  
endl;  
    exit(1);  
}
```

Go

```
groups, next_page_token, total_size, err := client.GetGroups(  
    []string{"group1_id"}, 5, "")  
if err != nil {  
    log.Fatalf("Error getting groups: %v", err)  
}
```

Java

```
PaginatedResult<Group> result =  
client.getGroups(Arrays.asList("group1_id"), 5, "");  
// result.value is a list of groups  
// result.nextToken is next page token  
// result.totalSize is total size
```

Node

```
const [groups, nextPageToken, totalSize] = await client.getGroups(  
    ["group1_id"],  
    5,  
    ""  
);
```

Python

```
groups, next_page_token, total_size =  
client.get_groups(["group1_id"], 5, "")
```

Ruby

```
groups, next_page_token, total_size = client.get_groups(  
    ids: ["group1_id"], page_size: 5, page_token: ''  
)
```

 For a tutorial on pagination, see [here](#) ↗.

Delete Group

Delete Group deletes a group. This operation also disassociates identities from the group, but doesn't delete any identities.

C#

```
int DeleteGroup(string id)
```

```
async Task<int> DeleteGroupAsync(string id)
```

C++

```
paravision::Status DeleteGroup(int &rows_affected, const string &id)
```

Go

```
DeleteGroup(id string) (int, error)
```

Java

```
int deleteGroup(String id) throws ClientException
```

Node

```
async deleteGroup(id: string) => int
```

Python

```
delete_group(id: str) -> int
```

Ruby

```
delete_group(id) -> Integer
```

Arguments

id: A string, the group ID.

Return

rows_affected: The number of groups removed as a result of this request.

status: C++, [Status](#) object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var rowsAffected = client.DeleteGroup("group_id");
```

C++

```
int rows_affected;  
auto status = client.DeleteGroup(rows_affected, "group_id");  
if(!status.ok())  
{  
    cerr << "Error deleting group! " << status.error_message <<  
endl;  
    exit(1);  
}
```

Go

```
rowsAffected, err := client.DeleteGroup("group_id")  
if err != nil {  
    log.Fatalf("Error deleting group: %v", err)  
}
```

Java

```
int rowsAffected = client.deleteGroup("group_id");
```

Node

```
var rowsAffected = await client.deleteGroup("group_id");
```

Python

```
rows_affected = client.delete_group("group_id")
```

Ruby

```
rows_affected = client.delete_group('group_id')
```

Update Group

Update Group updates a group with a new name.

C#

```
pb.Group UpdateGroup(string id, string name)
```

```
async Task<pb.Group> UpdateGroupAsync(string id, string name)
```

C++

```
paravision::Status UpdateGroup(Group &group, const string &id,  
const string &name)
```

Go

```
UpdateGroup(id string, name string) (*pb.Group, error)
```

Java

```
Group updateGroup(String id, String name) throws ClientException
```

Node

```
async updateGroup(id: string, name: string) => !Group
```

Python

```
update_group(id: str, name: str) -> Group
```

Ruby

```
update_group(id, name) -> Group
```

Arguments

id: A string, the group ID.

name: A string, the new name of the group.

Return

group: A [Group](#), the updated group.

status: C++, [Status](#) object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var group = client.UpdateGroup("group_id", "admin");
```

C++

```
Group group;
auto status = client.UpdateGroup(group, "group_id", "admin");
if(!status.ok())
{
    cerr << "Error updating group! " << status.error_message <<
endl;
    exit(1);
}
```

Go

```
group, err := client.UpdateGroup("group_id", "admin")
if err != nil {
    log.Fatalf("Error updating group: %v", err)
}
```

Java

```
Group group = client.updateGroup("group_id", "admin");
```

Node

```
var group = await client.updateGroup("group_id", "admin");
```

Python

```
group = client.update_group("group_id", "admin")
```

Ruby

```
group = client.update_group('group_id', 'admin')
```

Add Face

Add Face adds a face to an identity.

C#

```
pb.Face AddFace(string identityId, IEnumerable<float> embedding,  
    float threshold, float quality, string model = "",  
    float scalingFactor = 2.0f,  
    int bucketsLimit = 32,  
    bool flush = true)
```

```
async Task<pb.Face> AddFaceAsync(string identityId,  
    IEnumerable<float> embedding,  
    float threshold, float quality, string model = "",  
    float scalingFactor = 2.0f,  
    int bucketsLimit = 32,  
    bool flush = true)
```

C++

```
paravision::Status AddFace(Face &face, const string &identity_id,  
    const Embedding &embedding,  
    const float threshold,  
    const float quality,  
    const string &model = "",  
    const float scaling_factor = 2.0f,  
    const int buckets_limit = 32,  
    bool flush = true)
```

Go

```
AddFace(identityId string, embedding []float32, threshold float32,  
    quality float32, model string,  
    scalingFactor float32, bucketsLimit int, flush bool) (*pb.Face,  
    error)
```

Java

```
Face addFace(  
    String identityId,  
    List<Float> embedding,  
    float threshold,  
    float quality,  
    String model = "",  
    float scalingFactor = 2,  
    int bucketsLimit = 32,  
    boolean flush = true  
) throws ClientException
```

Node

```
async addFace(  
    identityId: string,  
    embedding: (!Embedding|Array<float>),  
    threshold: float,  
    quality: float,  
    model: string="",  
    scalingFactor: float = 2.0,  
    bucketsLimit: int = 32,  
    flush: Boolean = true) => !Face
```

Python

```
add_face(identity_id: str, embedding: Embedding, threshold: float,  
quality: float, model: str = "",  
scaling_factor: float = 2.0, buckets_limit: int = 32, flush: bool =  
True) -> Face
```

Ruby

```
add_face(identity_id, embedding, threshold, quality, model: '',  
scaling_factor: 2.0, buckets_limit: 32, flush: true)  
-> Face
```

Arguments

identity_id: The ID of the identity that the new face belongs to.

embedding: An [embedding](#) of the new face. This embedding can be generated using our [Recognition SDK ↗](#) and our [Processor API ↗](#).

threshold: A float representing a valid [score](#) value. If nonzero, the operation will not succeed (an error will be returned) unless the given embedding matches the identity with a score greater than this threshold.

quality: A float between 0 and 1, the quality score of the embedding.

model: A string that represents the model used to generate the embedding. This argument can be left blank if the user does not wish to specify a model name.

scaling_factor: A float greater than 1, optional. Rate at which number of considered embeddings is increased while searching for already-existing embedding for **threshold** check.

buckets_limit: An integer, optional. Corresponds to `nprobe` parameter from [Milvus ↗](#), used while searching for already-existing embedding for **threshold** check.

flush: A bool, optional. When true, newly added embeddings will be immediately considered in search operations at the cost of slowing down this insert operation.



scaling_factor, **buckets_limit** and **flush** are only used when Milvus is configured as a database.

Return

face: A [Face](#), the new face.

status: C++, [Status](#) object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var face = client.AddFace("identity_id", embedding, 0.5, 0.9, "tarka");
```

C++

```
Face face;
auto status = client.AddFace(face, "identity_id", embedding, 0.5, 0.9, "tarka");
if(!status.ok())
{
    cerr << "Error adding face! " << status.error_message << endl;
    exit(1);
}
```

Go

```
face, err := client.AddFace("identity_id", embedding, 0.5, 0.9, "tarka")
if err != nil {
    log.Fatalf("Error adding face: %v", err)
}
```

Java

```
Face face = client.addFace("identity_id", embedding, 0.5, 0.9, "tarka");
```

Node

```
var face = await client.addFace("identity_id", embedding, 0.5, 0.8, "tarka");
```

Python

```
face = client.add_face("identity_id", embedding, 0.5, 0.8, "tarka")
```

Ruby

```
face = client.add_face('identity_id', embedding, 0.5, 0.8, model: 'tarka')
```

Add Faces

Add Faces adds multiple faces to an identity.

C#

```
List<pb.Face> AddFaces(string identityId,
    IEnumerable<IEnumerable<float>> embeddings,
    float threshold,
    IEnumerable<float> qualities,
    string model = "",
    float scalingFactor = 2.0f,
    int bucketsLimit = 32,
    bool flush = true)
```

```
async Task<List<pb.Face>> AddFacesAsync(string identityId,  
    IEnumerable<IEnumerable<float>> embeddings,  
    float threshold,  
    IEnumerable<float> qualities,  
    string model = "",  
    float scalingFactor = 2.0f,  
    int bucketsLimit = 32,  
    bool flush = true)
```

C++

```
paravision::Status AddFaces(vector<Face> &faces, const string  
&identity_id,  
    const vector<Embedding> &embeddings,  
    const float threshold,  
    const vector<float> &qualities,  
    const string &model = "",  
    const float scaling_factor = 2.0f,  
    const int buckets_limit = 32,  
    bool flush = true)
```

Go

```
AddFaces(id string, embeddings [][]float32, threshold float32,  
    qualities []float32, model string,  
    scalingFactor float32, bucketsLimit int, flush bool) ([]*pb.Face,  
    error)
```

Java

```
List<Face> addFaces(
String identityId,
List<List<Float>> embeddings,
float threshold,
List<Float> qualities,
String model = "",
float scalingFactor = 2,
int bucketsLimit = 32,
boolean flush = true
) throws ClientException
```

Node

```
async addFaces(
identityId: string,
embeddings: Array<(!Embedding|Array<float>>>,
threshold: float,
qualities: Array<float>
model: string = "",
scalingFactor: float = 2.0,
bucketsLimit: int = 32,
flush: Boolean = true) => Array<!Face>
```

Python

```
add_faces(identity_id: str, embeddings: List[Embedding],
threshold: float, qualities: List[float], model: str = "",
scaling_factor: float = 2.0, buckets_limit: int = 32, flush: bool =
True)
-> List[Face]
```

Ruby

```
add_faces(identity_id, embeddings, threshold, qualities, model: '',
scaling_factor: 2.0, buckets_limit: 32, flush: true)
-> Array[Face]
```

Arguments

identity_id: The ID of the identity that the new face belongs to.

embeddings: A list of [embeddings](#), one for each face to add to the identity. These embeddings can be generated using our [Recognition SDK ↗](#) and our [Processor API ↗](#).

threshold: A float representing a valid [score](#) value. If nonzero, the operation will not succeed (an error will be returned) unless all embeddings match the identity with a score greater than this threshold.

qualities: A list of floats between 0 and 1, the quality scores of the embedding.

model: A string that represents the model used to generate the embedding. This argument can be left blank if the user does not wish to specify a model name.

scaling_factor: A float greater than 1, optional. Rate at which number of considered embeddings is increased while searching for already-existing embedding for **threshold** check.

buckets_limit: An integer, optional. Corresponds to `nprobe` parameter from [Milvus ↗](#), used while searching for already-existing embedding for **threshold** check.

flush: A bool, optional. When true, newly added embeddings will be immediately considered in search operations at the cost of slowing down this insert operation.



scaling_factor, **buckets_limit** and **flush** are only used when Milvus is configured as a database.

Return

faces: List of [Faces](#), the new faces.

status: C++, `Status` object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var faces = client.AddFaces("identity_id", embeddings, 0.5,
    qualities, "tarka");
```

C++

```
vector<Face> faces;
auto status = client.AddFaces(
    faces, "identity_id", embeddings, 0.5, qualities, "tarka");
if(!status.ok())
{
    cerr << "Error adding faces! " << status.error_message << endl;
    exit(1);
}
```

Go

```
faces, err := client.AddFaces(
    "identity_id", embeddings, 0.5, qualities, "tarka")
if err != nil {
    log.Fatalf("Error adding faces: %v", err)
}
```

Java

```
List<Face> faces = client.addFaces(
    "identity_id", embeddings, 0.5, qualities, "tarka");
```

Node

```
var faces = await client.addFaces(  
  "identity_id",  
  embeddings,  
  0.5,  
  qualities,  
  "tarka"  
);
```

Python

```
faces = client.add_faces("identity_id", embeddings, 0.5, qualities,  
"tarka")
```

Ruby

```
faces = client.add_faces(  
  'identity_id', embeddings, 0.5, qualities, model: 'tarka'  
)
```

Get Face

Get Face retrieves a face.

C#

```
pb.Face GetFace(string id)
```

```
async Task<pb.Face> GetFaceAsync(string id)
```

C++

```
paravision::Status GetFace(Face &result, const string &id)
```

Go

```
GetFace(id string) (*pb.Face, error)
```

Java

```
Face getFace(String id) throws ClientException
```

Node

```
async getFace(id: string) => !Face
```

Python

```
get_face(id: str) -> Face
```

Ruby

```
get_face(id) -> Face
```

Arguments

id: A string, the face ID.

Return

face: A `Face`, the requested face.

status: C++, `Status` object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var face = client.GetFace("face_id");
```

C++

```
Face face;
auto status = client.GetFace(face, "face_id");
if(!status.ok())
{
    cerr << "Error getting face! " << status.error_message << endl;
    exit(1);
}
```

Go

```
face, err := client.GetFace("face_id")
if err != nil {
    log.Fatalf("Error getting face: %v", err)
}
```

Java

```
Face face = client.getFace("face_id");
```

Node

```
const face = await client.getFace("face_id");
```

Python

```
face = client.get_face("face_id")
```

Ruby

```
face = client.get_face('face')
```

Get Faces

Get faces retrieves the faces belonging to an identity.

C#

```
Tuple<List<pb.Face>, string, int> GetFaces(string identityId,  
    int pageSize = 100,  
    string pageToken = "")
```

```
async Task<Tuple<List<pb.Face>, string, int>> GetFacesAsync(string  
identityId,  
    int pageSize = 100,  
    string pageToken = "")
```

C++

```
paravision::Status GetFaces(tuple<vector<Face>, string, int>
&result,
const string &identity_id,
const int page_size = 100,
const string &page_token = "")
```

Go

```
GetFaces(identityId string, pageSize int, pageToken string)
([]*pb.Face, string, int, error)
```

Java

```
PaginatedResult<Face> getFaces(
String identityId,
int pageSize = 100,
String pageToken = ""
) throws ClientException
```

Node

```
async getFaces(
identityId: string,
pageSize: int = 100,
pageToken: string = "") => [Array<!Face>, string, int]
```

Python

```
get_faces(identity_id: str, page_size: int = 100, page_token: str =
"")
-> Tuple[List[Face], str, int]
```

Ruby

```
get_faces(identity_id, page_size: 100, page_token: '')  
-> [Array[Face], str, int]
```

Arguments

identity_id: A string, the ID of the identity to retrieve faces from.

page_size: An integer, the number of results to return. Defaults to 100.

page_token: A string specifying the page of results to return. Using the default empty string will return the first page of results. Subsequent calls should use the returned `next_page_token` for the value of this parameter.

Return

faces: List of `Faces`, the faces belonging to the identity.

next_page_token: A string, the value of the page token associated with the next page of results. If this is an empty string, there are no more results.

total_size: An integer, the total count of faces belonging to the identity.

status: C++, `Status` object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var result = client.GetFaces("identity_id", 5, "");  
// result.Item1 is list of faces  
// result.Item2 is next page token  
// result.Item3 is total size
```

C++

```
tuple<vector<Face>, string, int> result;  
auto status = client.GetFaces(result, "identity_id", 5, "");  
if(!status.ok())  
{  
    cerr << "Error getting faces! " << status.error_message << endl;  
    exit(1);  
}
```

Go

```
faces, next_page_token, total_size, err := client.GetFaces(  
    "identity_id", 5, "")  
if err != nil {  
    log.Fatalf("Error getting faces: %v", err)  
}
```

Java

```
PaginatedResult<Face> result = client.getFaces("identity_id", 5,  
    "");  
// result.value is list of faces  
// result.nextToken is next page token  
// result.totalSize is total size
```

Node

```
const [faces, nextPageToken, totalSize] = await client.getFaces(  
  "identity_id",  
  5,  
  ""  
);
```

Python

```
faces, next_page_token, total_size = client.get_faces(  
  "identity_id",  
  5,  
  ""  
)
```

Ruby

```
faces, next_page_token, total_size = client.get_faces(  
  'identity_id', page_size: 5, page_token: ''  
)
```

 For a tutorial on pagination, see [here](#).

Delete Face

Delete Face deletes a face. This operation will fail if it would cause the identity to have no faces remaining.

C#

```
int DeleteFace(string identityId, string faceId)
```

```
async Task<int> DeleteFaceAsync(string identityId, string faceId)
```

C++

```
paravision::Status DeleteFace(int &rows_affected, const string  
&identity_id, const string &face_id)
```

Go

```
DeleteFace(identityId string, faceId string) (int, error)
```

Java

```
int deleteFace(  
String identityId,  
String faceId,  
) throws ClientException
```

Node

```
async deleteFace(identityId: string, faceId: string) => int
```

Python

```
delete_face(identity_id: str, face_id: str) -> int
```

Ruby

```
delete_face(identity_id, face_id) -> Integer
```

Arguments

identity_id: A string, the ID of the identity of the face to delete.

face_id: A string, the ID of the face to delete.

Return

rows_affected: The number of faces removed as a result of this request.

status: C++, [Status](#) object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var rowsAffected = client.DeleteFace("identity_id", "face_id");
```

C++

```
int rows_affected;  
auto status = client.DeleteFace(rows_affected, "identity_id",  
                                "face_id");  
if(!status.ok())  
{  
    cerr << "Error deleting face! " << status.error_message << endl;  
    exit(1);  
}
```

Go

```
rowsAffected, err := client.DeleteFace("identity_id", "face_id")
if err != nil {
    log.Fatalf("Error deleting face: %v", err)
}
```

Java

```
int rowsAffected = client.deleteFace("identity_id", "face_id");
```

Node

```
var rowsAffected = await client.deleteFace("identity_id",
"face_id");
```

Python

```
rows_affected = client.delete_face("identity_id", "face_id")
```

Ruby

```
rows_affected = client.delete_face('identity_id', 'face_id')
```

Delete Faces

Delete Faces deletes faces belonging to an identity. This operation will fail if it would cause the identity to have no faces remaining.

C#

```
int DeleteFaces(string identityId, IEnumerable<string> faceIds)
```

```
async Task<int> DeleteFacesAsync(string identityId,  
IEnumerable<string> faceIds)
```

C++

```
paravision::Status DeleteFaces(int &rows_affected, const string  
&identity_id,  
const vector<string> &face_ids)
```

Go

```
DeleteFaces(identityId string, faceIds []string) (int, error)
```

Java

```
int deleteFaces(  
String identityId,  
List<String> faceIds,  
) throws ClientException
```

Node

```
async deleteFaces(identityId: string, faceIds: Array<string>) => int
```

Python

```
delete_faces(identity_id: str, face_ids: List[str]) -> int
```

Ruby

```
delete_faces(identity_id, face_ids) -> Integer
```

Arguments

identity_id: A string, the ID of the identity of the faces to delete.

face_ids: A list of strings, the ids of the faces to delete.

Return

rows_affected: The number of faces removed as a result of this request.

status: C++, [Status](#) object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
var rowsAffected = client.DeleteFaces("identity_id", new  
List<string>{"face1_id", "face2_id"});
```

C++

```
int rows_affected;
auto status = client.DeleteFaces(rows_affected, "identity_id",
{"face1_id", "face2_id"});
if(!status.ok())
{
    cerr << "Error deleting faces! " << status.error_message <<
endl;
    exit(1);
}
```

Go

```
rowsAffected, err := client.DeleteFaces("identity_id",
[]string{"face1_id", "face2_id"})
if err != nil {
    log.Fatalf("Error deleting faces: %v", err)
}
```

Java

```
int rowsAffected = client.deleteFaces("identity_id",
Arrays.asList("face1_id", "face2_id"));
```

Node

```
var rowsAffected = await client.deleteFaces("identity_id", [
    "face1_id",
    "face2_id",
]);
```

Python

```
rows_affected = client.delete_faces("identity_id", ["face1_id",
"face2_id"])
```

Ruby

```
rows_affected = client.delete_faces('identity_id', ['face1_id',  
'face2_id'])
```

Compact Database

 Available only when Milvus is used as a database.

Clean up deleted (or leftover) entities, potentially improving query performance.

Normally, Milvus only performs soft-delete of embeddings, which causes use of more and more disk space and decrease in speed of search operations.

Additionally, in rare cases, failed inserts and deletes may cause lingering Milvus embeddings that don't correspond to any identity. This operation forces removal of all unnecessary entities.

 **Note:** Even when `compact` operation is called, Milvus will execute it only if more than 10% of embeddings have been soft-deleted. As such, consider calling this method only after a significant amount of deletions or periodically during low load.

C#

```
void CompactDatabase()
```

```
async Task CompactDatabaseAsync()
```

C++

```
paravision::Status CompactDatabase()
```

Go`CompactDatabase() error`**Java**`void compactDatabase() throws ClientException`**Node**`async compactDatabase() => void`**Python**`compact_database() -> None`**Ruby**`compact_database() -> nil`

Return

status: C++, [Status](#) object indicating call status. If no errors occurred, `status.ok()` will be `true`.

Example

C#

```
client.CompactDatabase();
```

C++

```
auto status = client.CompactDatabase();  
if(!status.ok())  
{  
    cerr << "Error while compacting! " << status.error_message <<  
endl;  
    exit(1);  
}
```

Go

```
err := client.CompactDatabase()  
if err != nil {  
    log.Fatalf("Error while compacting: %v", err)  
}
```

Java

```
client.compactDatabase();
```

Node

```
await client.compactDatabase();
```

Python

```
client.compact_database()
```

Ruby

```
client.compact_database()
```

Health Check

Query health of the Identity API.

Identity API implements [gRPC Health checking protocol's](#) `Check` endpoint, it's available as a standard `/grpc.health.v1.Health/Check` RPC method.

To be used by infrastructure components like load balancers for purposes of monitoring the health of the container or for horizontal scaling.

Arguments

service: A string, ignored. Identity API exposes only one service, all checks return server's overall health status.

Return

status: An enum of type `ServingStatus`. It can have one of three possible values, `UNKNOWN`, `SERVING`, or `NOT_SERVING`.

Example

grpc_health_probe

[grpc_health_probe](#) is available inside the container for easy setup:

```
docker exec <IDENTITY_CONTAINER_NAME> /bin/grpc_health_probe -  
addr=localhost:5656
```

gRPCurl

[gRPCurl](#) ↗ can be used to query the health:

```
grpcurl -plaintext localhost:5656 grpc.health.v1.Health/Check
```

HTTP usage

Setup

The Identity API container is capable of exposing an HTTP interface for all its features. The HTTP server is not enabled by default, you must set the `PV_HTTP_INTERFACE` environment variable of the Identity API container to `on` to activate it. You can do this e.g. by adding `-e PV_HTTP_INTERFACE=on` to the `docker run` command when starting the container. Additionally, you must expose the HTTP port to outside of the container, which can be done with `-p 8080:8080`. Thus, to use the HTTP interface, you need to start the Identity container like this:

```
docker run -dt --rm --name identity-api --link <DB_CONTAINER_NAME> -e
PV_HTTP_INTERFACE=on -p 5656:5656 -p 8080:8080 \
    -e PV_SCORING_METRIC=match_score -e
PV_POSTGRES_URI=postgres://<DB_USER>:<DB_PASSWORD>@<DB_CONTAINER_NAME>:
<DB_PORT>/<DB_NAME>?sslmode=disable \
    containers.paravision.ai/identity/identity:<tag>
```

Refer to [installation](#) for details on starting processor container.

CORS

Cross-origin resource sharing (CORS) can be enabled for the HTTP server by setting the `PV_CORS_ORIGINS` environment variable. CORS is disabled by default. To enable it, set the `PV_CORS_ORIGINS` environment variable to a comma separated list of origins that should be allowed. For origin values, `*` and `null` can also be used. For example, `-e PV_CORS_ORIGINS=*` can be used with the `docker run` command when starting the container to enable CORS for all origins.

HTTP basic authentication

[HTTP basic authentication ↗](#) can be enabled for the HTTP server. It is disabled by default. To enable it, provide a username and a password in `PV_USER` and `PV_PASSWORD` environment variables. There is also `PV_PASSWORD_FILE` environment variable available if you wish to use a file within the container instead of an environment variable to provide the password to the HTTP server (like, for example, a Docker secret).

You can test authentication with `curl` by providing a `-u USER:PASSWORD` flag to the command. For example:

```
curl -u USER:PASSWORD -s -X GET http://localhost:8080/v7/identity | jq .
```

If credentials are invalid or missing, the server will respond with status code `401 Unauthorized` and one of the following message:

```
{
  "code": 16,
  "message": "invalid user or password",
  "details": []
}
```

or

```
{
  "code": 16,
  "message": "lacking authorization header",
  "details": []
}
```

Common features

Unless otherwise stated, all arguments for GET and DELETE requests need to be provided as query parameters.

Unless otherwise stated, all arguments for POST requests need to be passed as JSON objects (`application/json`). URL-encoded form fields (`application/x-www-form-urlencoded`) are not supported.

Data is returned in response body as JSON-formatted objects (`application/json`).

All endpoints indicate errors with a relevant HTTP response code, and additionally a message describing error details is available as a JSON-formatted object in response body, with following fields:

message: A human-readable description of the error.

code: An internal error type number.

details: Reserved for future use.

An example error response looks like this:

```
{
  "message": "parsing field \"image\": illegal base64 data at input
byte 462",
  "code": 3,
  "details": []
}
```

Lookup

POST /v7/lookup

Lookup finds identities that closely match each of the given embeddings.

Request parameters

embeddings: A list of embeddings to lookup. These embeddings can be generated using our [Recognition SDK ↗](#) and our [Processor API ↗](#).

group_ids: A list of group ids. The identity search will be limited to identities belonging to these groups.

limit: An integer, the maximum number of identities returned per embedding. Default is 1.

model: A string that represents the model used to generate the embedding. This argument can be left blank if the user does not wish to specify a model name.

scaling_factor: A float greater than 1, optional. Rate at which number of considered embeddings is increased while searching for requested number of identities. Defaults to 2.0.

buckets_limit: An integer, optional. Corresponds to `nprobe` parameter from [Milvus ↗](#), used while searching for requested number of identities. Defaults to 32.

 **scaling_factor, buckets_limit** are only used when Milvus is configured as a database.

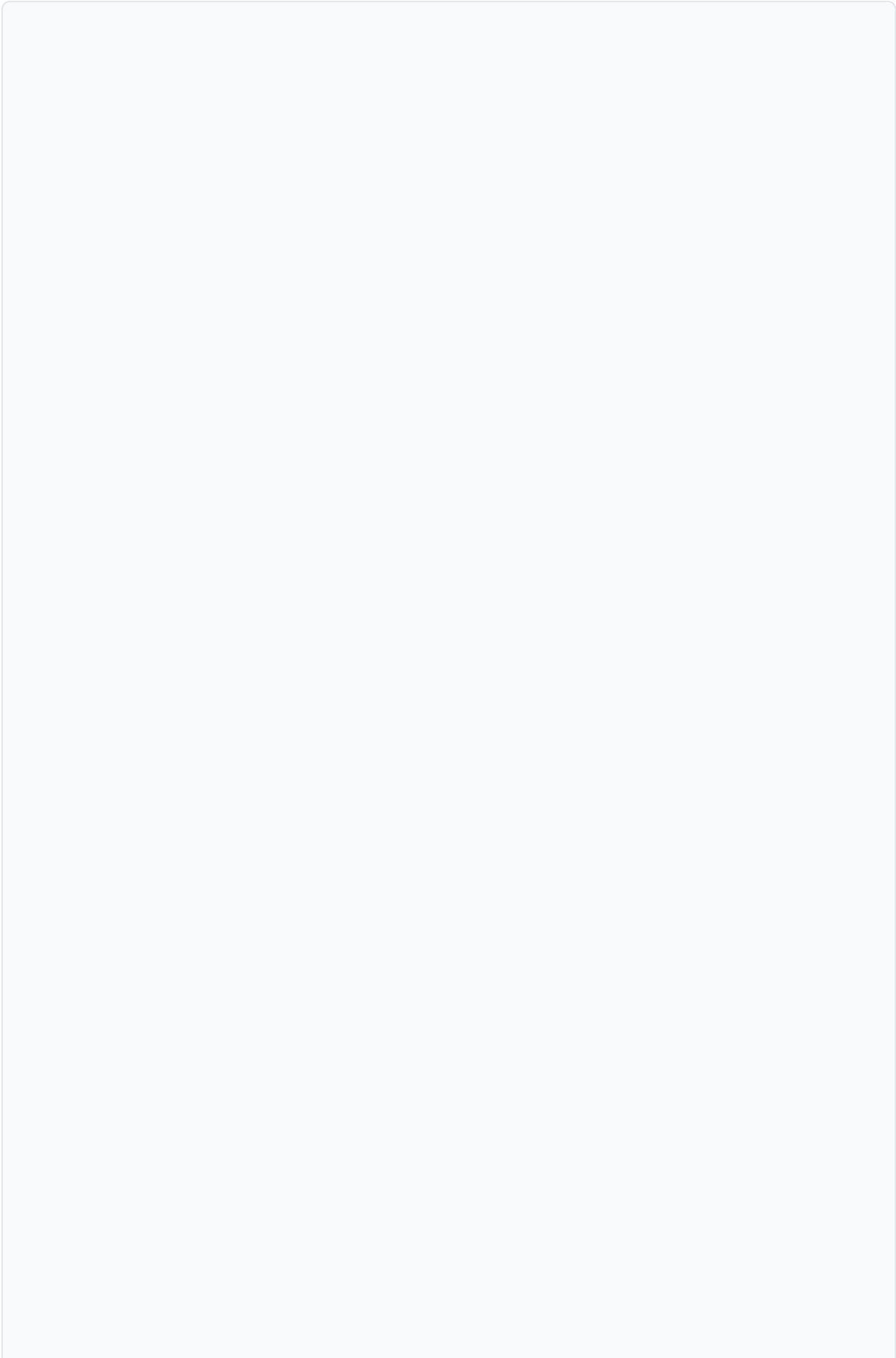
Response

A single JSON object with the following field:

lookup_identities: A list of objects, one per each embedding, that contain lookup results. Each object has a single field "matches", which is a list of up to `limit` (or less) [IdentityMatches](#). These, in turn, contain both an identity and a score of a potential match. The [score](#) represents how similar the embedding is to the closest one belonging to the identity.

Example

```
echo '{
  "embeddings": [
    {
      "embedding": [
        0.52348,
        -0.99235,
        0.210091,
        ...
      ]
    },
    {
      "embedding": [
        -0.91123,
        -0.80942,
        0.11203,
        ...
      ]
    }
  ],
  "limit": 2
}' | curl -s -X POST http://localhost:8080/v7/lookup -d@- | jq .
```



```

{
  "lookup_identities": [
    {
      "matches": [
        {
          "identity": {
            "id": "i_c00q7jlti3v8r1c91p4g",
            "created_at": "2021-01-15T14:16:46.976552Z",
            "updated_at": "2021-01-15T14:16:46.976552Z",
            "group_ids": [],
            "external_id": ""
          },
          "score": 0.7512235515
        },
        {
          "identity": {
            "id": "i_c00q7jlti3v8r1c91p20",
            "created_at": "2021-01-15T14:16:46.963837Z",
            "updated_at": "2021-01-15T14:16:46.963837Z",
            "group_ids": [
              "g_c00q7jlti3v8r1c91p10"
            ],
            "external_id": ""
          },
          "score": 0.0216810468
        }
      ]
    },
    {
      "matches": [
        {
          "identity": {
            "id": "i_c00q7jlti3v8r1c91p4g",
            "created_at": "2021-01-15T14:16:46.976552Z",
            "updated_at": "2021-01-15T14:16:46.976552Z",
            "group_ids": [],
            "external_id": ""
          },
          "score": 0.99935473998
        },
        {
          "identity": {
            "id": "i_c00q7jlti3v8r1c91pag",
            "created_at": "2021-01-15T14:16:46.995694Z",
            "updated_at": "2021-01-15T14:16:46.995694Z",
            "group_ids": [
              "g_c00q7jlti3v8r1c91p10"
            ],

```

```
        "external_id": ""
      },
      "score": 3.6014888e-05
    }
  ]
}
]
```

Create Identities

POST /v7/identities

Create Identities registers new identities.

Request parameters

embeddings: A list of embeddings, each one corresponding to the face of a new identity. These embeddings can be generated using our [Recognition SDK ↗](#) and our [Processor API ↗](#).

threshold: A float representing a valid [score](#) value. If nonzero, the operation will not succeed (an error will be returned) if at least one of the given embeddings matches an already-existing embedding in the database with a score greater than this threshold.

qualities: A list of floats between 0 and 1, the quality scores of the embeddings.

group_ids: A list, containing the IDs of any groups that the new identities belong to.

model: A string that represents the models used to generate the embeddings. This argument can be left blank if the user does not wish to specify a model name.

external_ids: List of strings, optional. One per every identity created. Any custom value to be stored alongside each corresponding identity. Can optionally be used when searching for an identity.

scaling_factor: A float greater than 1, optional. Rate at which number of considered embeddings is increased while searching for already-existing embedding for **threshold** check. Defaults to 2.0.

buckets_limit: An integer, optional. Corresponds to `nprobe` parameter from [Milvus](#), used while searching for already-existing embedding for **threshold** check. Defaults to 32.

options: A list of `CreateIdentitiesOptions`, additional options allowing change of default behavior.

 **scaling_factor** and **buckets_limit** are only used when Milvus is configured as a database.

Response

A single JSON object with the following field:

identities: List of `Identities`, the new identities.

Example

```
echo '{
  "embeddings": [
    {
      "embedding": [
        0.52348,
        -0.99235,
        0.210091,
        ...
      ]
    },
    {
      "embedding": [
        -0.91123,
        -0.80942,
        0.11203,
        ...
      ]
    }
  ],
  "threshold": 0.75,
  "qualities": [
    0.8,
    0.9
  ],
  "group_ids": [
    "g_c00q7jlti3v8r1c91p10"
  ],
  "external_ids": [
    "953711",
    "511263"
  ]
}' | curl -s -X POST http://localhost:8080/v7/identities -d@- | jq .
```

```
{
  "identities": [
    {
      "id": "i_c02pd15ti3v7jamcen9g",
      "created_at": "2021-01-15T14:09:08.616460Z",
      "updated_at": "2021-01-15T14:09:08.616460Z",
      "group_ids": [
        "g_c00q7jlti3v8r1c91p10"
      ],
      "external_id": "953711"
    },
    {
      "id": "i_c02pd15ti3v7jamcenag",
      "created_at": "2021-01-15T14:09:08.616460Z",
      "updated_at": "2021-01-15T14:09:08.616460Z",
      "group_ids": [
        "g_c00q7jlti3v8r1c91p10"
      ],
      "external_id": "511263"
    }
  ]
}
```

Get Identity

```
GET /v7/identity
```

Get identity retrieves an identity.

Request parameters

id: A string, the identity ID.

external_id: A string, optional. If provided, `id` becomes optional, and only an identity with a matching `external_id` will be returned.

If both `id` and `external_id` are used, this function will search for an identity that matches both criteria.

Response

A single JSON object with the following field:

identity: An `Identity`, the requested identity.

Example

```
curl -s -X GET -G http://localhost:8080/v7/identity \
  -d id=i_c00q7jlti3v8r1c91p4g | jq .
```

```
{
  "identity": {
    "id": "i_c00q7jlti3v8r1c91p4g",
    "created_at": "2021-01-15T14:16:46.976552Z",
    "updated_at": "2021-01-15T14:16:46.976552Z",
    "group_ids": [
      "g_c00q7jlti3v8r1c91p10"
    ],
    "external_id": ""
  }
}
```

Get Identities

```
GET /v7/identities
```

Get identities retrieves a set of identities.

Request parameters

group_ids: A list of group ids. The identity search will be limited to identities belonging to these groups. If this is empty, identities will be searched across all

groups. Default is an empty list.

page_size: An integer, the number of results to return. Defaults to 100.

page_token: A string specifying the page of results to return. Using the default empty string will return the first page of results. Subsequent calls should use the returned `next_page_token` for the value of this parameter.

Response

A single JSON object with the following fields:

identities: List of `Identities`, the retrieved identities.

next_page_token: A string, the value of the page token associated with the next page of results. If this is an empty string, there are no more results.

total_size: An integer, the total count of identities that belong to the groups.

Example

```
curl -s -X GET -G http://localhost:8080/v7/identities \
  -d page_size=2 \
  --data-urlencode page_token=xgMAAA== \
  -d group_ids=g_c00q7jlti3v8r1c91p10 | jq .
```

```
{
  "identities": [
    {
      "id": "i_c00q7jlti3v8r1c91p4g",
      "created_at": "2021-01-15T14:16:46.976552Z",
      "updated_at": "2021-01-15T14:16:46.976552Z",
      "group_ids": [
        "g_c00q7jlti3v8r1c91p10"
      ],
      "external_id": ""
    },
    {
      "id": "i_c00q7jlti3v8r1c91p70",
      "created_at": "2021-01-15T14:16:46.987551Z",
      "updated_at": "2021-01-15T14:16:46.987551Z",
      "group_ids": [
        "g_c00q7jlti3v8r1c91p10"
      ],
      "external_id": ""
    }
  ],
  "next_page_token": "yAMAAA==",
  "total_size": 6
}
```

Delete Identities

```
DELETE /v7/identities
```

Delete identities deletes a set of identities.

Request parameters

ids: A list of strings, the identity ids.

external_ids: A list of strings, the external identity ids.

Response

A single JSON object with the following field:

rows_affected: The number of identities removed as a result of this request.

Example

```
curl -s -X DELETE -G http://localhost:8080/v7/identities \
  -d ids=i_c00q7jlti3v8r1c91p70 | jq .
```

```
{
  "rows_affected": 1
}
```

Update Identity Groups

```
POST /v7/update_identity_groups
```

Update Identity Groups adds and removes an identity from specified groups.

Request parameters

id: A string, the ID of the identity to update.

add_group_ids: A list of strings, the ids of the groups to add the identity to. Default: empty.

remove_group_ids: A list of strings, the ids of the groups to remove the identity from. Default: empty.

flush: A bool, optional. When true, identity will be immediately available in search operations with new groups at the cost of slowing down this update operation. Defaults to true.

 **flush** is only used when Milvus is configured as a database.

Response

A single JSON object with the following field:

identity: An `Identity`, the updated identity.

Example

```
echo '{
  "id": "i_c00q7jlti3v8r1c91p4g",
  "remove_group_ids": ["g_c00q7jlti3v8r1c91p10"]
}' | curl -s -X POST http://localhost:8080/v6/update_identity_groups
-d@- | jq .
```

```
{
  "identity": {
    "id": "i_c00q7jlti3v8r1c91p4g",
    "created_at": "2021-01-15T14:16:46.976552Z",
    "updated_at": "2021-01-15T14:16:46.976552Z",
    "group_ids": [],
    "external_id": ""
  }
}
```

Create Group

```
POST /v7/group
```

Create Group creates a new group.

Request parameters

name: A string, the name of the new group.

Response

A single JSON object with the following field:

group: A [Group](#), the new group.

Example

```
echo '{
  "name": "My Group"
}' | curl -s -X POST http://localhost:8080/v7/group -d@- | jq .
```

```
{
  "group": {
    "id": "g_c02o7olti3v7jamcen90",
    "name": "My Group",
    "created_at": "2021-01-15T12:49:38.814873Z",
    "updated_at": "2021-01-15T12:49:38.814873Z"
  }
}
```

Get Group

```
GET /v7/group
```

Get Group retrieves a group.

Request parameters

id: A string, the group ID.

Response

A single JSON object with the following field:

group: A `Group`, the group.

Example

```
curl -s -X GET -G http://localhost:8080/v7/group \
  -d id=g_c02o7olti3v7jamcen90 | jq .
```

```
{
  "group": {
    "id": "g_c02o7olti3v7jamcen90",
    "name": "My Group",
    "created_at": "2021-01-15T12:49:38.814873Z",
    "updated_at": "2021-01-15T12:49:38.814873Z"
  }
}
```

Get Groups

```
GET /v7/groups
```

Get groups retrieves a set of groups.

Request parameters

ids: A list of group ids. If this list is empty or `None`, all groups will be searched. Default is an empty list.

page_size: An integer, the number of results to return. Defaults to 100.

page_token: A string specifying the page of results to return. Using the default empty string will return the first page of results. Subsequent calls should use the returned `next_page_token` for the value of this parameter.

Response

A single JSON object with the following fields:

groups: List of `Groups`, the retrieved groups.

next_page_token: A string, the value of the page token associated with the next page of results. If this is an empty string, there are no more results.

total_size: An integer, the total count of groups.

Example

```
curl -s -X GET -G http://localhost:8080/v7/groups \
  -d page_size=2 \
  --data-urlencode page_token=kAIAAA== | jq .
```

```
{
  "groups": [
    {
      "id": "g_c00q7jlti3v8r1c91p10",
      "name": "My Group",
      "created_at": "2021-01-15T14:16:46.954846Z",
      "updated_at": "2021-01-15T14:16:46.958237Z"
    },
    {
      "id": "g_c00q7jlti3v8r1c91p1g",
      "name": "Another group",
      "created_at": "2021-01-15T14:16:46.956844Z",
      "updated_at": "2021-01-15T14:16:46.956844Z"
    }
  ],
  "next_page_token": "kAIAAA==",
  "total_size": 3
}
```

Delete Group

```
DELETE /v7/group
```

Delete Group deletes a group. This operation also disassociates identities from the group, but doesn't delete any identities.

Request parameters

id: A string, the group ID.

Response

A single JSON object with the following field:

rows_affected: The number of groups removed as a result of this request.

Example

```
curl -s -X DELETE -G http://localhost:8080/v7/group \  
-d id=g_c02o7olti3v7jamcen90 | jq .
```

```
{  
  "rows_affected": 1  
}
```

Update Group

```
POST /v7/update_group
```

Update Group updates a group with a new name.

Request parameters

id: A string, the group ID.

name: A string, the new name of the group.

Response

A single JSON object with the following field:

group: A [Group](#), the updated group.

Example

```
echo '{
  "id": "g_c00q7jlti3v8r1c91p10",
  "name": "New Group Name"
}' | curl -s -X POST http://localhost:8080/v7/update_group -d@- | jq
.
```

```
{
  "group": {
    "id": "g_c00q7jlti3v8r1c91p10",
    "name": "New Group Name",
    "created_at": "2021-01-15T14:16:46.954846Z",
    "updated_at": "2021-01-15T13:19:42.781011Z"
  }
}
```

Add Faces

```
POST /v7/faces
```

Add Faces adds multiple faces to an identity.

Request parameters

identity_id: The ID of the identity that the new face belongs to.

embeddings: A list of embeddings, one for each face to add to the identity. These embeddings can be generated using our [Recognition SDK ↗](#) and our [Processor API ↗](#).

threshold: A float representing a valid [score](#) value. If nonzero, the operation will not succeed (an error will be returned) unless all embeddings match the identity with a score greater than this threshold.

qualities: A list of floats between 0 and 1, the quality scores of the embedding.

model: A string that represents the model used to generate the embedding. This argument can be left blank if the user does not wish to specify a model name.

scaling_factor: A float greater than 1, optional. Rate at which number of considered embeddings is increased while searching for already-existing embedding for **threshold** check. Defaults to 2.0.

buckets_limit: An integer, optional. Corresponds to `nprobe` parameter from [Milvus](#) [↗](#), used while searching for already-existing embedding for **threshold** check. Defaults to 32.

flush: A bool, optional. When true, newly added embeddings will be immediately considered in search operations at the cost of slowing down this insert operation. Defaults to true.

 **scaling_factor**, **buckets_limit** and **flush** are only used when Milvus is configured as a database.

Response

A single JSON object with the following field:

faces: List of [Faces](#), the new faces.

Example

```

echo '{
  "identity_id": "i_c02pd15ti3v7jamcen9g",
  "embeddings": [
    {
      "embedding": [
        0.52348,
        -0.99235,
        0.210091,
        ...
      ]
    },
    {
      "embedding": [
        0.51132,
        -0.89332,
        0.211101,
        ...
      ]
    }
  ],
  "threshold": 0,
  "qualities": [
    0.8,
    0.9
  ]
}' | curl -s -X POST http://localhost:8080/v7/faces -d@- | jq .

```

```

{
  "faces": [
    {
      "id": "f_c02phdtti3v7jamcencg",
      "identity_id": "i_c02pd15ti3v7jamcen9g",
      "created_at": "2021-01-15T14:18:31.852330Z",
      "model": "",
      "quality": 0.8
    },
    {
      "id": "f_c02phdtti3v7jamcend0",
      "identity_id": "i_c02pd15ti3v7jamcen9g",
      "created_at": "2021-01-15T14:18:31.852330Z",
      "model": "",
      "quality": 0.9
    }
  ]
}

```

Get Face

```
GET /v7/face
```

Get Face retrieves a face.

Request parameters

id: A string, the face ID.

Response

A single JSON object with the following fields:

face: A [Face](#), the requested face.

Example

```
curl -s -X GET -G http://localhost:8080/v6/face \
  -d id=f_c02pd15ti3v7jamcena0 | jq .
```

```
{
  "face": {
    "id": "f_c02pd15ti3v7jamcena0",
    "identity_id": "i_c02pd15ti3v7jamcen9g",
    "created_at": "2021-01-15T14:09:08.616460Z",
    "model": "",
    "quality": 0.8
  }
}
```

Get Faces

```
GET /v7/faces
```

Get faces retrieves the faces belonging to an identity.

Request parameters

identity_id: A string, the ID of the identity to retrieve faces from.

page_size: An integer, the number of results to return. Defaults to 100.

page_token: A string specifying the page of results to return. Using the default empty string will return the first page of results. Subsequent calls should use the returned `next_page_token` for the value of this parameter.

Response

A single JSON object with the following fields:

faces: List of `Faces`, the faces belonging to the identity.

next_page_token: A string, the value of the page token associated with the next page of results. If this is an empty string, there are no more results.

total_size: An integer, the total count of faces belonging to the identity.

Example

```
curl -s -X GET -G http://localhost:8080/v7/faces \
  -d identity_id=i_c02pd15ti3v7jamcen9g \
  -d page_size=2 \
  --data-urlencode page_token=xgMAAA== | jq .
```

```
{
  "faces": [
    {
      "id": "f_c02pd15ti3v7jamcena0",
      "identity_id": "i_c02pd15ti3v7jamcen9g",
      "created_at": "2021-01-15T14:09:08.616460Z",
      "model": "",
      "quality": 0.8
    },
    {
      "id": "f_c02phdtti3v7jamcencg",
      "identity_id": "i_c02pd15ti3v7jamcen9g",
      "created_at": "2021-01-15T14:18:31.852330Z",
      "model": "",
      "quality": 0.8
    }
  ],
  "next_page_token": "2QUAAA==",
  "total_size": 3
}
```

Delete Faces

```
DELETE /v7/faces
```

Delete Faces deletes faces belonging to an identity. This operation will fail if it would cause the identity to have no faces remaining.

Request parameters

identity_id: A string, the ID of the identity of the face to delete.

face_ids: A list of strings, the IDs of the faces to delete.

Response

A single JSON object with the following field:

rows_affected: The number of groups removed as a result of this request.

Example

```
curl -s -X DELETE -G http://localhost:8080/v7/faces \
  -d identity_id=i_c02pd15ti3v7jamcen9g \
  -d face_ids=f_c02pd15ti3v7jamcena0 \
  -d face_ids=f_c02phdtti3v7jamcencg | jq .
```

```
{
  "rows_affected": 2
}
```

Compact Database

```
GET /v7/compact_database
```

 Available only when Milvus is used as a database.

Clean up deleted (or leftover) entities, potentially improving query performance.

Normally, Milvus only performs soft-delete of embeddings, which causes use of more and more disk space and decrease in speed of search operations.

Additionally, in rare cases, failed inserts and deletes may cause lingering Milvus embeddings that don't correspond to any identity. This operation forces removal of all unnecessary entities.

 **Note:** Even when `compact` operation is called, Milvus will execute it only if more than 10% of embeddings have been soft-deleted. As such, consider calling this method only after a significant amount of deletions or periodically during low load.

Response

A single empty JSON object.

Example

```
curl -s -X GET -G http://localhost:8080/v7/compact_database | jq .
```

```
{}
```

Health Check

```
GET /v7/health_check
```

Query health of the Identity API.

This endpoint exposes [gRPC Health checking protocol's](#) `Check` method.

To be used by infrastructure components like load balancers for purposes of monitoring the health of the container or for horizontal scaling.

Arguments

service: A string, ignored. Identity API exposes only one service, all checks return server's overall health status.

Response

A single JSON object with the following field:

status: An enum of type `ServingStatus`. It can have one of three possible values, `UNKNOWN`, `SERVING`, or `NOT_SERVING`.

Example

```
curl -s -X GET http://localhost:8080/v7/health_check | jq .
```

```
{  
  "status": "SERVING"  
}
```

Types

The Identity API deals with 3 main types: `Identities`, `Faces`, and `Groups`. Additionally, the `Lookup` function exposes a type `IdentityMatch`.

A detailed description of these objects and their attributes and accessors is given below:

Create Identity Options/Create Identities Options

An enum providing the list of options to pass into the `Create Identity` and `Create Identities` function. Below are the possible values and explanations.

- **NO_FLUSH** - this option disables immediate data flushing, which improves insert performance at the cost of operation results being visible in lookups with a few seconds lag (**Milvus database only**).
- **THRESHOLD_WITHIN_GROUPS** - this option enables running `threshold` check against only specified groups (in `group_ids`) instead of all enrolled Identities.

Embedding

Embedding is a vector or a list of floats representing the face. It can be generated using our [Recognition SDK ↗](#) and our [Processor API ↗](#).

C#

```
IEnumerable<float>
```

C++

```
typedef std::vector<float> Embedding;
```

Go

```
[]float32
```

Java

```
List<Float>
```

Node

```
Array<float>
```

Python

```
List[float]
```

Ruby

```
# only arrays containing floats are accepted  
Array
```

Face

A Face object, belonging to an identity.

- **id:** A string, the unique identifier of this Face.

C#

```
var id = face.Id;
```

C++

```
string id = face.id;
```

Go

```
id := face.GetId()
```

Java

```
String id = face.getId();
```

JSON

```
face["id"]
```

Node

```
var id = face.getId();
```

Python

```
id = face.id
```

Ruby

```
id = face.id
```

- **identity_id**: A string, the unique identifier of the Identity the face belongs to.

C#

```
var identityId = face.IdentityId;
```

C++

```
string identity_id = face.identity_id;
```

Go

```
identityId := face.GetIdentityId()
```

Java

```
String identityId = face.getIdentityId();
```

JSON

```
face["identity_id"]
```

Node

```
var identityId = face.getIdentityId();
```

Python

```
identity_id = face.identity_id
```

Ruby

```
identity_id = face.identity_id
```

- **created_at:** A `Timestamp`, the time at which the Face was created.

C#

```
var createdAt = face.CreatedAt;
```

C++

```
auto created_at = face.created_at;
```

Go

```
createdAt := face.GetCreatedAt()
```

Java

```
var createdAt = face.getCreatedAt();
```

JSON

```
face["created_at"]
```

Node

```
var createdAt = face.getCreatedAt();
```

Python

```
created_at = face.created_at
```

Ruby

```
created_at = face.created_at
```

- **model:** A string, the model used to generate the embedding for the Face.

C#

```
var model = face.Model
```

C++

```
string model = face.model;
```

Go

```
model := face.GetModel()
```

Java

```
String model = face.getModel();
```

JSON

```
face["model"]
```

Node

```
var model = face.getModel();
```

Python

```
model = face.model
```

Ruby

```
model = face.model
```

- **quality:** A float between 0 and 1, the quality score for the face.

C#

```
float quality = face.Quality;
```

C++

```
float quality = face.quality;
```

Go

```
quality := face.GetQuality()
```

Java

```
float quality = face.getQuality();
```

JSON

```
face["quality"]
```

Node

```
var quality = face.getQuality();
```

Python

```
quality = face.quality
```

Ruby

```
quality = face.quality
```

Group

A Group object, to which multiple Identities can belong.

- **id**: A string, the unique identifier of this Group.

C#

```
var id = group.Id;
```

C++

```
string id = group.id;
```

Go

```
id := group.GetId()
```

Java

```
var id = group.getId();
```

JSON

```
group["id"]
```

Node

```
var id = group.getId();
```

Python

```
id = group.id
```

Ruby

```
id = group.id
```

- **name:** A string, the name of this group (e.g. `admin`).

C#

```
var name = group.Name;
```

C++

```
string name = group.name;
```

Go

```
name := group.GetName()
```

Java

```
String name = group.getName();
```

JSON

```
group["name"]
```

Node

```
var name = group.getName();
```

Python

```
name = group.name
```

Ruby

```
name = group.name
```

- **created_at:** A [Timestamp](#), the time at which the Group was created.

C#

```
var createdAt = group.CreatedAt;
```

C++

```
auto created_at = group.created_at;
```

Go

```
createdAt := group.GetCreatedAt()
```

Java

```
var createdAt = group.getCreatedAt();
```

JSON

```
group["created_at"]
```

Node

```
var createdAt = group.getCreatedAt();
```

Python

```
created_at = group.created_at
```

Ruby

```
created_at = group.created_at
```

- **updated_at:** A [Timestamp](#), the time at which the Group was last updated.

C#

```
var updatedAt = group.UpdatedAt;
```

C++

```
auto updated_at = group.updated_at;
```

Go

```
updatedAt := group.GetUpdatedAt()
```

Java

```
var updatedAt = group.getUpdatedAt();
```

JSON

```
group["updated_at"]
```

Node

```
var updatedAt = group.getUpdatedAt();
```

Python

```
updated_at = group.updated_at
```

Ruby

```
updated_at = group.updated_at
```

IdentityMatch

An IdentityMatch represents an `Identity` and associated score of the match.

- **identity:** An `Identity`

C#

```
var identity = match.Identity;
```

C++

```
auto identity = match.identity;
```

Go

```
identity := match.GetIdentity()
```

Java

```
Identity identity = match.getIdentity();
```

JSON

```
match["identity"]
```

Node

```
var identity = match.getIdentity();
```

Python

```
identity = match[0]  
# alternatively:  
identity, _ = match
```

Ruby

```
identity = match.identity
```

- **score:** A float, the score of the match to the identity

C#

```
float score = match.Score;
```

C++

```
float score = match.score;
```

Go

```
score := match.GetScore()
```

Java

```
float score = match.getScore();
```

JSON

```
match["score"]
```

Node

```
var score = match.getScore();
```

Python

```
score = match[1]  
# alternatively:  
_, score = match
```

Ruby

```
score = match.score
```

Identity

An Identity represents a single person.

- **id**: A string, the unique identifier of this Identity.

C#

```
var id = identity.Id;
```

C++

```
string id = identity.id;
```

Go

```
id := identity.GetId()
```

Java

```
var id = identity.getId();
```

JSON

```
identity["id"]
```

Node

```
var id = identity.getId();
```

Python

```
id = identity.id
```

Ruby

```
id = identity.id
```

- **created_at:** A `Timestamp`, the time at which the Identity was created.

C#

```
var createdAt = identity.CreatedAt;
```

C++

```
auto created_at = identity.created_at;
```

Go

```
createdAt := identity.GetCreatedAt()
```

Java

```
var createdAt = identity.getCreatedAt();
```

JSON

```
identity["created_at"]
```

Node

```
var createdAt = identity.getCreatedAt();
```

Python

```
created_at = identity.created_at
```

Ruby

```
created_at = identity.created_at
```

- **updated_at:** A [Timestamp](#), the time at which the Identity was last updated.

C#

```
var updatedAt = identity.UpdatedAt;
```

C++

```
auto updated_at = identity.updated_at;
```

Go

```
updatedAt := identity.GetUpdatedAt()
```

Java

```
var updatedAt = identity.getUpdatedAt();
```

JSON

```
identity["updated_at"]
```

Node

```
var updatedAt = identity.getUpdatedAt();
```

Python

```
updated_at = identity.updated_at
```

Ruby

```
updated_at = identity.updated_at
```

- **group_ids:** A list of strings, the IDs of the groups this Identity belongs to.

C#

```
var groupIds = identity.GroupIds;
```

C++

```
vector<string> group_ids = identity.group_ids;
```

Go

```
groupIds := identity.GetGroupIds()
```

Java

```
var groupIds = identity.getGroupIds();
```

JSON

```
identity["group_ids"]
```

Node

```
var groupIds = identity.getGroupIds();
```

Python

```
group_ids = identity.group_ids
```

Ruby

```
group_ids = identity.group_ids
```

- **external_id:** A user-defined extra value stored alongside this identity, which can be used to reference an entry in an external database.

C#

```
var externalId = identity.ExternalId;
```

C++

```
string external_id = identity.external_id;
```

Go

```
externalId := identity.GetExternalId()
```

Java

```
var externalId = identity.getExternalId();
```

JSON

```
identity["external_id"]
```

Node

```
var externalId = identity.getExternalId();
```

Python

```
external_id = identity.external_id
```

Ruby

```
external_id = identity.external_id
```

PaginatedResult (Java)

A dataclass that gathers paginated query return values into a single structure. This data type is returned by functions that return data grouped into pages.

```
public class PaginatedResult<V> {  
    public final List<V> value;  
    public final String nextToken;  
    public final int totalSize;  
}
```

- **value** - A list of some of the returned values (a single page).
- **nextToken** - The token for accessing the next page with a subsequent call to the same function.
- **totalSize** - The total count of all values (sum of all page sizes).

For detailed semantics of the returned value, please refer to the documentation of a specific function that returns a `PaginatedResult`.

Status (C++)

A Status object in C++ representing call status (modeled after `grpc::Status` [↗](#), extracted not to require gRPC headers installed). See [Errors](#) for more details on possible error conditions.

- **code** - A `StatusCode`, numerical value representing error (or lack thereof)
- **error_message** - A string, human-readable error message
- **error_details** - A string, additional raw error details from gRPC
- **ok()** - A function, returning boolean depending on the status (`true` if successful), example usage:

```
if(!status.ok())
{
    cerr << "Error: " << status.error_message << endl;
    exit(1);
}
```

StatusCode (C++)

An enum providing statuses for `Status` object

- **OK**
- **CANCELLED**
- **UNKNOWN**
- **INVALID_ARGUMENT**
- **DEADLINE_EXCEEDED**
- **NOT_FOUND**
- **ALREADY_EXISTS**
- **PERMISSION_DENIED**
- **RESOURCE_EXHAUSTED**
- **FAILED_PRECONDITION**
- **ABORTED**
- **OUT_OF_RANGE**
- **UNIMPLEMENTED**
- **INTERNAL**
- **UNAVAILABLE**
- **DATA_LOSS**
- **UNAUTHENTICATED**

Timestamp

A timestamp is used to represent time (from [Protobuf](#) ↗).

Note: In JSON format, the Timestamp type does not contain any fields, and is instead a plain string, with an ISO 8601 representation of the timestamp (`YYYY-mm-ddTHH:MM:SS.ffffffZ`).

- **seconds:** Represents seconds of UTC time since Unix epoch.

C#

```
var seconds = timestamp.Seconds;
```

C++

```
auto seconds = timestamp.seconds;
```

Go

```
seconds := timestamp.GetSeconds()
```

Java

```
var seconds = timestamp.getSeconds();
```

Node

```
var seconds = timestamp.getSeconds();
```

Python

```
seconds = timestamp.seconds
```

Ruby

```
seconds = timestamp.seconds
```

- **nanos:** Non-negative fractions of a second at nanosecond resolution.

C#

```
var nanos = timestamp.Nanos;
```

C++

```
auto nanos = timestamp.nanos;
```

Go

```
nanos := timestamp.GetNanos()
```

Java

```
var nanos = timestamp.getNanos();
```

Node

```
var nanos = timestamp.getNanos();
```

Python

```
nanos = timestamp.nanos
```

Ruby

```
nanos = timestamp.nanos
```

Errors

Error Responses

This document identifies the errors that Identity API returns.

In most languages the errors are passed-through in exceptions but in some (like C++ and Go) they might be present in return values instead.

gRPC Errors

API extends [gRPC status codes](#) with their error messages describing the failure details.

Status Code	Short Description
CANCELLED (number: 1 , retryable: ☒ , HTTP Status Code: 408 Request Timeout)	The operation was cancelled
UNKNOWN (number: 2 , retryable: ☒ , HTTP Status Code: 500 Internal Server Error)	Unknown error
INVALID_ARGUMENT (number: 3 , retryable: ☐ , HTTP Status Code: 400 Bad Request)	The client specified an invalid argument
DEADLINE_EXCEEDED (number: 4 , retryable: ☒ , HTTP Status Code: 504 Gateway Timeout)	The deadline expired before the operation could complete
NOT_FOUND (number: 5 , retryable: ☐ , HTTP Status Code: 404 Not Found)	Some requested entity was not found
ALREADY_EXISTS (number: 6 , retryable: ☐ , HTTP Status Code: 409 Conflict)	The entity that a client attempted to create already exists

PERMISSION_DENIED (number: 7 , retryable: no , HTTP Status Code: 403 Forbidden)	The caller does not have permission to execute the specified operation
RESOURCE_EXHAUSTED (number: 8 , retryable: yes , HTTP Status Code: 429 Too Many Requests)	Some resource has been exhausted
FAILED_PRECONDITION (number: 9 , retryable: no , HTTP Status Code: 400 Bad Request)	The operation was rejected because the system is not in a state required for the operation's execution
ABORTED (number: 10 , retryable: yes , HTTP Status Code: 409 Conflict)	The operation was aborted
OUT_OF_RANGE (number: 11 , retryable: no , HTTP Status Code: 400 Bad Request)	The operation was attempted past the valid range
UNIMPLEMENTED (number: 12 , retryable: no , HTTP Status Code: 501 Not Implemented)	The operation is not implemented or is not supported/enabled in this service
INTERNAL (number: 13 , retryable: yes , HTTP Status Code: 500 Internal Server Error)	Internal error
UNAVAILABLE (number: 14 , retryable: yes , HTTP Status Code: 503 Service Unavailable)	The service is currently unavailable
DATA_LOSS (number: 15 , retryable: no , HTTP Status Code: 500 Internal Server Error)	Unrecoverable data loss or corruption
UNAUTHENTICATED (number: 16 , retryable: no , HTTP Status Code: 401 Unauthorized)	The request does not have valid authentication credentials for the operation

HTTP Error Reporting

When using our API through HTTP Interface, gRPC status codes will be translated to corresponding HTTP status codes with error message details in the body as

JSON:

```
{  
  "message": "Error message",  
  "code": 13,  
  "details": []  
}
```

Troubleshooting

Invalid Argument

Invalid argument errors (like `Invalid input argument: name [details]`) describe issues with invalid argument(s) passed into the method call. You need to verify your code to check if values sent to the API actually conform to the [specifications](#).

Deadline Exceeded

Load balancers can inject this error message in case they have troubles connecting to our API servers. Check their logs and make sure API servers, load balancers and client applications can reach each other within your specified timeouts.

Failed Precondition

Thrown when operations are requested in a wrong order (usually like `Client initialization error: message`). This might happen when you forget to connect to the API server using `connect` before calling other methods.

Unimplemented

Thrown when trying to call a functionality which is not implemented, usually when using mismatched client/server versions (often like `Protocol version in`

`client(v1.0)` and `server(v2.1)` are incompatible). Make sure you are using compatible server and client libraries.

Unavailable

Generally returned by load balancers, clients, not our API. Usually means network connectivity issues. Check the logs and make sure API servers, load balancers are running and the servers are actually reachable.

Unauthenticated

Most likely authentication is enabled on the server but necessary credentials are not provided in the request metadata (or are invalid). Recheck your configuration.

Release Notes

API

v7.0.0

Aug 11, 2023

Functional Release Notes

- Updated `weight` and `bias` for option `PV_SCORING_METRIC=match_score` for Gen6 templates.
- **Note: do not run database migrations for this release while Identity API is up.** See Engineering Description for details.
- A performance improvement in paginated GetIdentities call, especially noticeable for very large numbers of identities.
- Minor improvement related to error messages and codes.
- The option `PV_SCORING_METRIC=confidence` is **removed**.

Engineering Description

- `total_size` returned by the GetIdentities call is now cached in a separate table in the SQL database, instead of being recomputed from scratch on every paginated query. **Please note that the migration introducing this cached value cannot be ran concurrent to inserting or deleting identities, otherwise you might end up with an inaccurate value.**
- There were various changes and improvements, both in clients and servers, related to error messages and returned gRPC error codes. See also the new [Errors](#) page in the documentation.
- Fixed below CVEs:
 - CVE-2021-43565
 - CVE-2022-27191
 - CVE-2021-33194
 - CVE-2022-27664
 - CVE-2022-41723

- CVE-2022-29526
- CVE-2021-38561
- CVE-2022-32149
- CVE-2022-27664
- CVE-2022-41721
- CVE-2022-41723
- CVE-2022-32149
- CVE-2020-8911
- CVE-2022-2582
- GHSA-76wf-9vgp-pj7w
- CVE-2020-8912
- CVE-2021-43565
- CVE-2022-27191
- CVE-2022-27664
- CVE-2022-41723
- CVE-2022-29526
- CVE-2021-38561
- CVE-2022-32149

C#**v7.0.0***Aug 11, 2023*

- Updated client lib for Identity API v7.0.0.

C++**v7.0.0***Aug 11, 2023*

- Updated client lib for Identity API v7.0.0.

Go

v7.0.0

Aug 11, 2023

- Updated client lib for Identity API v7.0.0.

HTTP

v7.0.0

Aug 11, 2023

- Updated client lib for Identity API v7.0.0.

Java

v7.0.0

Aug 11, 2023

- Updated client lib for Identity API v7.0.0.

Node

v7.0.0

Aug 11, 2023

- Updated client lib for Identity API v7.0.0.

Python

v7.0.0

Aug 11, 2023

- Updated client lib for Identity API v7.0.0.

Ruby

v7.0.0

Aug 11, 2023

- Updated client lib for Identity API v7.0.0.